

TravelHub — Backend

Guía de estudio: explicación del proyecto

Documento de repaso

6 de agosto de 2026

Objetivo: entender, como estudiante novato, el papel de cada archivo y carpeta del backend de TravelHub. Cada sección tiene explicación, analogía y código real del proyecto.

Índice

1. El contexto: estructura del backend	8
2. requirements.txt — la lista de la compra	8
2.1. ¿Qué es?	8
2.2. Contenido y papel de cada librería	8
2.3. Cómo se conecta con el proyecto	9
3. venv — el entorno virtual	9
3.1. ¿Qué es?	9
3.2. ¿Por qué existe?	9
3.3. Cómo se usa	9
3.4. Regla de oro	9
4. __pycache__ — basura automática	10
4.1. ¿Qué es?	10
4.2. Datos rápidos	10
5. app — el código real	10
6. main.py — el punto de entrada	10
6.1. Los imports (líneas 1–6)	10
6.2. Crear y actualizar la base de datos (líneas 8–14)	11
6.3. Crear la aplicación (línea 16)	11
6.4. El vigilante CORS (líneas 18–24)	11
6.5. Montar las rutas (líneas 26–34)	12
6.6. La ruta raíz (líneas 36–38)	12
6.7. Resumen del flujo completo	12

7. config.py — la central de configuraciones	13
7.1. ¿Qué es?	13
7.2. Contenido completo (7 líneas)	13
7.3. Explicación concepto por concepto	13
7.3.1. <code>import os</code>	13
7.3.2. <code>os.getenv(nombre, valor_por_defecto)</code> — la estrella del archivo . . .	13
7.3.3. Las variables, una por una	13
7.3.4. ¿Por qué existe el “valor por defecto”?	14
7.3.5. ¿Y qué pasa en producción (el servidor)?	14
8. db/database.py — la conexión con la base de datos	14
8.1. ¿Qué es?	14
8.2. Contenido completo (19 líneas)	14
8.3. Explicación concepto por concepto	15
8.3.1. Los imports (líneas 1–4)	15
8.3.2. El detalle de SQLite (líneas 6–8)	15
8.3.3. <code>engine</code> (línea 10)	15
8.3.4. <code>SessionLocal</code> (línea 11)	15
8.3.5. <code>Base</code> (línea 12)	16
8.3.6. <code>get_db()</code> (líneas 14–19) — la “fábrica con limpieza”	16
9. db/__init__.py — el “carnet” de carpeta	16
9.1. ¿Qué es?	16
9.2. Para qué existe	16
9.2.1. ¿Y si está vacío? ¿Funciona?	17
10.models/ — las tablas de la base de datos	17
10.1. ¿Qué es <code>models/</code> ?	17
11.models/user.py — el modelo Usuario	17
11.1. Contenido completo (17 líneas)	17
11.2. Explicación concepto por concepto	18
11.2.1. Los imports (líneas 1–3)	18
11.2.2. La clase y su nombre de tabla (líneas 5–6)	18
11.2.3. Las columnas (líneas 8–17)	18
11.2.4. Los 3 niveles de “restricción” (clave para novatos)	19
11.2.5. La diferencia <code>default</code> vs <code>server_default</code>	19
11.2.6. ¿De dónde salen esas columnas extra de <code>main.py</code> ?	19
12.models/service.py — el modelo Servicio	19
12.1. Contenido completo (17 líneas)	19
12.2. ¿Qué es un <code>Service</code> ?	20
12.3. Lo nuevo: la columna con <code>ForeignKey</code> (línea 9)	20
12.4. Las columnas, una por una	20
12.4.1. El nuevo tipo <code>Float</code>	21
12.4.2. El patrón de las columnas <code>sin nullable=False</code>	21
13.models/booking.py — el modelo Reserva	21
13.1. Contenido completo (14 líneas)	21
13.2. ¿Qué es una <code>Booking</code> ?	22
13.3. Lo destacado: DOS claves foráneas (líneas 9–10)	22
13.4. Las columnas, una por una	22
13.5. El estado como texto (<code>status</code>) — una decisión de diseño	22

14.models/itinerary.py — el modelo Itinerario	23
14.1. Contenido completo (12 líneas)	23
14.2. ¿Qué es una <code>Itinerary</code> ?	23
14.3. Lo nuevo: el tipo <code>JSON</code> (línea 11)	23
14.3.1. <code>JSON</code> en una frase	23
14.4. Las columnas, una por una	24
14.5. El patrón que se repite	24
15.models/message.py — el modelo Mensaje	24
15.1. Contenido completo (14 líneas)	24
15.2. ¿Qué es una <code>Message</code> ?	25
15.3. Lo destacado: tres claves foráneas (líneas 9–11)	25
15.3.1. ¡Detalle curioso! Dos foráneas a la misma tabla	25
15.4. Las columnas, una por una	25
15.5. Cómo se conecta con <code>ws.py</code>	25
16.models/review.py — el modelo Reseña	26
16.1. Contenido completo (14 líneas)	26
16.2. ¿Qué es una <code>Review</code> ?	26
16.3. Lo destacado: tres claves foráneas (líneas 9–11)	26
16.3.1. La relación con <code>service.rating</code>	27
16.4. Las columnas, una por una	27
16.4.1. Detalle: <code>rating</code> es <code>Integer</code> aquí	27
17.models/__init__.py — el pegamento de los modelos	27
17.1. Contenido completo (6 líneas)	27
17.2. ¿Qué es?	28
17.3. La importación con punto (<code>from .user import User</code>)	28
17.3.1. ¿Para qué se hace esto?	28
17.3.2. El detalle CLAVE: ¿por qué importarlos aquí?	28
18.Resumen de models/ — las 6 tablas	29
19.schemas/ — los “formularios” de validación	29
19.1. ¿Qué es <code>schemas/</code> ?	29
19.2. ¿Por qué se necesitan? ¿No bastan los modelos?	29
19.3. Pydantic: el “inspector” de FastAPI	29
19.3.1. Los archivos de la carpeta	30
20.schemas/user.py — el formulario de registro	30
20.1. Contenido completo (7 líneas)	30
20.2. Explicación concepto por concepto	30
20.2.1. El <code>import</code> (línea 1)	30
20.2.2. La clase <code>UserCreate</code> (líneas 3–7)	31
20.2.3. Los “poderes” que da <code>BaseModel</code>	31
20.3. Comparación: <code>models/User</code> vs <code>schemas/UserCreate</code>	31
20.3.1. El detalle de la contraseña	31
21.schemas/token.py — los formularios del login	31
21.1. Contenido completo (9 líneas)	31
21.2. ¿Qué contiene? Dos clases en un solo archivo	32
21.3. <code>LoginRequest</code> — lo que el cliente envía	32
21.4. <code>Token</code> — lo que el servidor devuelve	32

21.5. Cómo funciona el login con estos dos formularios	32
21.5.1. El detalle de la validez del token	33
22.schemas/service.py — formularios de servicio	33
22.1. Contenido completo (17 líneas)	33
22.2. ¿Qué contiene? Dos formularios: crear y actualizar	33
22.3. <code>ServiceCreate</code> — crear un servicio	33
22.4. <code>ServiceUpdate</code> — actualizar un servicio	34
22.4.1. ¿Por qué <code>Optional</code> es importante aquí?	34
22.5. Los patrones de Pydantic que ya conocemos... y el nuevo	34
23.schemas/booking.py — formularios de reserva	34
23.1. Contenido completo (9 líneas)	34
23.2. ¿Qué contiene? Formularios de crear y cambiar estado	35
23.3. <code>BookingCreate</code> — reservar un servicio	35
23.3.1. El nuevo tipo <code>datetime</code>	35
23.3.2. ¿Qué no envía el cliente?	35
23.4. <code>BookingStatusUpdate</code> — cambiar el estado	35
23.5. Comparación rápida con el modelo <code>Booking</code>	35
24.schemas/itinerary.py — formularios del itinerario	36
24.1. Contenido completo (30 líneas)	36
24.2. ¿Qué contiene? Cinco clases (el archivo más completo)	37
24.3. <code>ItineraryCreate</code> y <code>ItineraryUpdate</code> — el plan diario	37
24.3.1. El nuevo tipo <code>Dict[str, Any]</code>	37
24.4. <code>RoutePoint</code> — un punto de la ruta (formulario anidado)	37
24.4.1. Los campos con valor por defecto	37
24.5. <code>DirectionsRequest</code> — pedir la ruta entre puntos	37
24.5.1. El nuevo tipo <code>List</code>	38
24.6. <code>DirectionsResponse</code> — la respuesta del mapa	38
24.7. Los formularios anidados (la idea clave del archivo)	38
25.schemas/message.py — formulario del mensaje	39
25.1. Contenido completo (7 líneas)	39
25.2. ¿Qué contiene? Un solo formulario	39
25.3. Lo que el cliente no envía	39
25.4. Cómo se conecta con <code>ws.py</code> y el modelo <code>Message</code>	39
26.schemas/review.py — formularios de la reseña	39
26.1. Contenido completo (20 líneas)	39
26.2. ¿Qué contiene? Un formulario de entrada y otro de salida	40
26.3. <code>ReviewCreate</code> — lo que envía el turista	40
26.4. <code>ReviewResponse</code> — lo que devuelve el servidor	40
26.4.1. La novedad: <code>class Config: from_attributes = True</code>	41
26.5. Los tres patrones de la carpeta <code>schemas/</code>	41
27.schemas/cost.py — el formulario de costos	41
27.1. Contenido completo (5 líneas)	41
27.2. ¿Qué contiene? Un solo formulario, un solo campo	42
27.3. ¿Para qué sirve?	42
27.4. ¿Por qué es una lista y no un número solo?	42
27.4.1. ¿Y por qué no se define la respuesta aquí?	42

28.Resumen de schemas/ — formularios de la API	42
29.routes/ — los “menús” de la API	43
29.1. ¿Qué es routes/?	43
29.2. ¿Cómo se ve una ruta?	43
30.routes/auth.py — la puerta de entrada (181 líneas)	43
30.1. ¿Qué hace este archivo?	43
30.2. Los imports (líneas 1–16)	44
30.3. La configuración (líneas 18–19)	44
30.4. Los mini-formularios locales (líneas 21–29)	44
30.5. Las funciones de ayuda (líneas 31–44)	45
30.5.1. Encriptar contraseñas: <code>hash_password</code>	45
30.5.2. ¿Por qué el “salt”?	45
30.5.3. Verificar contraseñas: <code>verify_password</code>	45
30.5.4. Crear el token: <code>create_access_token</code>	45
30.6. El registro: POST <code>/register</code> (líneas 46–76)	46
30.6.1. Los códigos HTTP más usados en este proyecto	46
30.7. Verificar email: POST <code>/verify</code> (líneas 78–99)	47
30.8. Reenviar código: POST <code>/resend-code</code> (líneas 101–116)	47
30.9. Iniciar sesión: POST <code>/login</code> (líneas 118–128)	47
30.10 Entrar con Google: POST <code>/google</code> (líneas 130–160)	48
30.11 La “máquina del carnet”: <code>get_current_user</code> (líneas 162–177)	48
30.11.1. ¿Cómo se usa en las demás rutas?	49
30.12 Mi perfil: GET <code>/me</code> (líneas 179–181)	49
30.13 Resumen del archivo	50
31.routes/services.py — el CRUD de servicios (93 líneas)	50
31.1. ¿Qué hace este archivo?	50
31.2. La configuración (línea 8)	50
31.3. Crear servicio: POST <code>/services</code> (líneas 10–29)	50
31.4. Listar y buscar: GET <code>/services</code> (líneas 31–56)	51
31.5. Ver un servicio: GET <code>/services/{id}</code> (líneas 58–63)	52
31.6. Actualizar: PUT <code>/services/{id}</code> (líneas 65–80)	52
31.7. Borrar: DELETE <code>/services/{id}</code> (líneas 82–93)	53
31.8. Los métodos HTTP que ya conocemos	53
32.routes/bookings.py — las reservas (77 líneas)	53
32.1. ¿Qué hace este archivo?	53
32.2. Crear reserva: POST <code>/bookings</code> (líneas 11–32)	54
32.3. Listar reservas: GET <code>/bookings</code> (líneas 34–42)	54
32.3.1. La novedad: <code>Booking.service_id.in(service_ids)</code>	55
32.4. Ver una reserva: GET <code>/bookings/{id}</code> (líneas 44–54)	55
32.5. Cambiar estado: PUT <code>/bookings/{id}/status</code> (líneas 56–77)	56
33.routes/itineraries.py — los planes de viaje (131 líneas)	57
33.1. ¿Qué hace este archivo?	57
33.2. CRUD básico (líneas 13–60)	57
33.3. Agregar una reserva al plan (líneas 62–79)	57
33.4. Quitar una reserva: POST <code>/itineraries/{id}/remove-booking/{booking_id}</code> (líneas 81–91)	58
33.5. Listar las reservas del plan: GET <code>/itineraries/{id}/bookings</code> (líneas 93–117)	58

33.5.1. El “join” a mano (la idea importante)	59
33.6. Calcular rutas: POST /itineraries/{id}/directions (líneas 119–131)	59
34.routes/messages.py — los mensajes HTTP (47 líneas)	60
34.1. ¿Qué hace este archivo?	60
34.2. Enviar mensaje: POST /messages (líneas 12–32)	60
34.3. Leer el historial: GET /messages/booking/{booking_id} (líneas 34–47)	61
34.3.1. La novedad: order_by	61
34.4. Mensajes HTTP vs WebSockets (recapitulemos)	61
35.routes/reviews.py — las calificaciones (59 líneas)	62
35.1. ¿Qué hace este archivo?	62
35.2. Crear reseña: POST /reviews (líneas 13–50)	62
35.2.1. La escalera de validaciones	63
35.2.2. Actualizar el promedio (líneas 45–48): la parte especial	63
35.3. Ver reseñas de un servicio: GET /reviews/service/{service_id} (líneas 52–55)	64
35.4. Ver mis reseñas: GET /reviews/my (líneas 57–59)	64
36.routes/costs.py — el calculador de costos (47 líneas)	64
36.1. ¿Qué hace este archivo?	64
36.2. El cálculo: POST /costs/calculate (líneas 11–47)	64
36.2.1. Las tres partes del resultado	65
36.2.2. El patrón if not ...: continue	65
36.2.3. El diccionario breakdown y su inicialización	66
36.2.4. La respuesta con diccionarios anidados (líneas 43–47)	66
37.routes/admin.py — el panel del administrador (39 líneas)	66
37.1. ¿Qué hace este archivo?	66
37.2. La guardia: require_admin (líneas 9–12)	67
37.3. La lista de espera: GET /prestadores-pendientes (líneas 14–17)	67
37.4. Aprobar: POST /aprobar/{user_id} (líneas 19–28)	68
37.5. Rechazar: POST /rechazar/{user_id} (líneas 30–39)	68
37.5.1. ¿Por qué “aprobar” solo marca y “rechazar” borra?	69
38.routes/ws.py — el chat en tiempo real (107 líneas)	69
38.1. ¿Qué hace este archivo?	69
38.2. Primero, lo importante: ¿qué es un WebSocket?	70
38.3. El ConnectionManager: la recepcionista (líneas 12–33)	70
38.4. El canal WebSocket (líneas 35–107)	71
38.4.1. Acción send_message: enviar (líneas 43–89)	72
38.4.2. La base de datos “a mano”: next(get_db()) y finally	73
38.4.3. Acción mark_read: marcar leído (líneas 91–104)	73
38.5. ¡Fin de routes/! El mapa de la carpeta	73
39.email_utils.py — el cartero (52 líneas)	74
39.1. ¿Qué hace este archivo?	74
39.2. La configuración del cartero (líneas 8–12)	74
39.3. El código de 6 dígitos (líneas 14–15)	75
39.4. El envío real: _send_email (líneas 17–49)	75
39.4.1. El modo simulado (líneas 19–21): el secreto para desarrolladores	75
39.4.2. Construir el correo (líneas 23–40)	76
39.4.3. Entregar el correo (líneas 42–45)	76
39.4.4. La red de seguridad (líneas 48–49)	76

39.5. La función que todos usan: <code>send_verification_email</code> (líneas 51–52)	77
40.maps_utils.py — el navegador (112 líneas)	77
40.1. ¿Qué hace este archivo?	77
40.2. Primero, ¿qué es una “API key”?	78
40.3. El plan A: <code>get_directions</code> (líneas 4–62)	78
40.3.1. Procesar la respuesta buena de Google (líneas 34–60)	79
40.4. El plan B: <code>_estimate_directions</code> (líneas 65–112)	79
40.5. La lección grande: el fallback (plan B)	80
41.travelhub.db — la base de datos (el archivo de datos)	81
41.1. ¿Qué es este archivo?	81
41.2. ¿Qué hay adentro? Las 6 tablas	81
41.3. La primera lección: <code>create_all</code> no toca lo que ya existe	82
41.4. La segunda lección: <code>*.db</code> está en <code>.gitignore</code>	82
42.Procfile — la instrucción para subir la app (1 línea)	83
42.1. ¿Para qué sirve?	83
42.2. Desmenucemos la línea	83
42.3. La conexión con <code>render.yaml</code> y <code>main.py</code>	84

1. El contexto: estructura del backend

Dentro de la carpeta **TravelHub** existen dos grandes carpetas: **backend** y **frontend**. En el backend encontramos:

Elemento	Rol general
requirements.txt	Lista de librerías del proyecto
venv	Entorno virtual (librerías aisladas)
__pycache__	Código compilado automático
app	El código real del backend
Procfile	Instrucción de arranque para el servidor
travelhub.db	Base de datos SQLite (local)

Puntos clave:

- `venv` y `__pycache__` están ignorados por git (`.gitignore`).
- No son código nuestro: `venv` se regenera, `__pycache__` se recrea solo.

2. requirements.txt — la lista de la compra

2.1 ¿Qué es?

Es el archivo que **declara todas las librerías** que necesita el backend para funcionar, con sus **versiones exactas**. Se instala todo de una vez con:

```
pip install -r requirements.txt
```

Analogía

Es como la **lista de la compra de un restaurante**: sin ella nadie sabría qué ingredientes hay que comprar antes de abrir.

2.2 Contenido y papel de cada librería

Librería	Versión	Para qué sirve
fastapi	0.139.2	El “cerebro” del backend: crea la API y sus rutas.
uvicorn	0.51.0	El “motor” que arranca y ejecuta FastAPI.
sqlalchemy	2.0.51	ORM: traduce Python a consultas de base de datos.
pydantic	2.13.4	Valida que los datos que entran/salen tengan formato correcto.
pydantic_core	2.46.4	Motor interno de pydantic (no se usa directamente).
python-jose	3.5.0	Crea y verifica tokens JWT (login).
passlib + bcrypt	1.7.4 / 5.0.0	Encriptan las contraseñas de los usuarios.
python-multipart	0.0.32	Permite recibir formularios y archivos.
email-validator	2.3.0	Valida el formato de los correos.
psycopg2-binary	2.9.12	Conexión con PostgreSQL (producción).
starlette	1.3.1	Marco base sobre el que se construye FastAPI.
pyjwt	2.13.0	Otra librería de JWT para firmar tokens.
cryptography	49.0.0	Utilidades de cifrado de bajo nivel.

Librería	Versión	Para qué sirve
requests	2.32.3	Cliente HTTP para llamar a otros servidores.

2.3 Cómo se conecta con el proyecto

Cadena de dependencia:

```
requirements.txt → venv (dónde se instala) → app (lo usa) → base de datos (SQLAlchemy)
```

Dato importante: las versiones exactas (==) no son casualidad. Por ejemplo, `passlib` es antigua y se lleva mal con `bcrypt 4.x+`. Cambiar una versión puede **romper el login**.

3. venv — el entorno virtual

3.1 ¿Qué es?

`venv` = entorno **venv** + **virtual**. Es una **cocina privada** por proyecto: dentro se guardan las librerías instaladas, aisladas del resto del sistema.

3.2 ¿Por qué existe?

Sin aislamiento, dos proyectos con versiones distintas de la misma librería se romperían entre sí. Con `venv`, cada proyecto tiene **su propio Python** y **sus propias librerías**.

Analogía

Cada proyecto es una casa y `venv` es su **cocina privada**, con sus propios ingredientes, sin compartir con las casas vecinas.

3.3 Cómo se usa

El backend se arranca con el Python *de dentro* de `venv`:

```
venv\Scripts\python.exe -m uvicorn app.main:app
```

3.4 Regla de oro

Resumen mental

`venv` **no se comparte ni se edita a mano**. Si se daña, se borra y se recrea:

```
python -m venv venv
pip install -r requirements.txt
```

4. `__pycache__` — basura automática

4.1 ¿Qué es?

Carpeta que genera Python al ejecutar código `.py`: guarda el **bytecode** (versión compilada) en archivos `.pyc` para no tener que volver a traducir el código cada vez.

```
app\main.py -> app\__pycache__\main.cpython-xxx.pyc
```

4.2 Datos rápidos

- ¿Lo escribo yo? No, Python lo genera solo.
- ¿Lo necesito? No, es una optimización automática.
- ¿Si lo borro? No pasa nada: se regenera en la siguiente ejecución.
- ¿Por qué `__pycache__`? Los dobles guiones (`....`, “dunder”) son la convención de Python para nombres internos especiales.

Resumen mental

Es **basura inofensiva**: no la edites, no la necesites, no la extrañes. Está ignorada por git (`.gitignore`).

5. `app` — el código real

Dentro de `app` encontramos las subcarpetas `db`, `models`, `routes` y `schemas`, más archivos de configuración y utilidades. Empezamos por `main.py` porque es el **punto de entrada**.

Analogía

`main.py` es la **puerta principal de un edificio**: si la abres primero, entiendes en qué habitaciones se divide la casa y cómo se conectan.

6. `main.py` — el punto de entrada

6.1 Los imports (líneas 1–6)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import text
from app.db.database import engine, Base
from app.models import User, Service, Booking, Itinerary, Message,
    Review
from app.routes import auth, services, bookings, itineraries, messages,
    reviews, costs, ws, admin
```

- FastAPI y CORSMiddleware: el motor y el “vigilante” de peticiones externas.
- `text`: permite ejecutar SQL en texto plano.

- **engine** y **Base**: el “cable” de conexión a la base de datos y la plantilla de la que derivan las tablas.
- **models**: las tablas (usuarios, servicios, reservas, itinerarios, mensajes, reseñas).
- **routes**: los “menús” de la API (login, servicios, reservas, etc.).

Analogía

Los imports son como **invitar a los trabajadores del restaurante** antes de abrir.

6.2 Crear y actualizar la base de datos (líneas 8–14)

```
Base.metadata.create_all(bind=engine)

with engine.connect() as conn:
    conn.execute(text("ALTER TABLE users ADD COLUMN IF NOT EXISTS
        verification_code VARCHAR"))
    conn.execute(text("ALTER TABLE users ADD COLUMN IF NOT EXISTS
        verification_code_expires TIMESTAMP"))
    conn.execute(text("ALTER TABLE users ADD COLUMN IF NOT EXISTS
        approved BOOLEAN DEFAULT FALSE"))
    conn.commit()
```

- **create_all**: crea todas las tablas **si no existen**. Por eso hay que importar los modelos antes (los “conoce” todos).
- **with engine.connect() as conn**:: abre la conexión, trabaja y la cierra sola.
- **ALTER TABLE ... IF NOT EXISTS**: “parche” de migraciones: agrega columnas nuevas a la tabla **users** si aún no las tiene (campos añadidos después).
- **conn.commit()**: “guarda” los cambios.

6.3 Crear la aplicación (línea 16)

```
app = FastAPI(title="TravelHub API")
```

Nace el objeto **app**, el “cerebro” que escucha las peticiones. El título es decorativo (aparece en la documentación automática).

6.4 El vigilante CORS (líneas 18–24)

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CORS = *Cross-Origin Resource Sharing*. Los navegadores bloquean peticiones entre dominios distintos por seguridad. Como el frontend (React) y el backend (FastAPI) viven en orígenes distintos, hay que **abrir la puerta**:

- `allow_origins=["*"]`: permite cualquier origen (abierto).
- `allow_methods=["*"]`: todos los métodos HTTP (GET, POST, PUT, DELETE...).
- `allow_headers=["*"]`: todos los encabezados.

6.5 Montar las rutas (líneas 26–34)

```
app.include_router(auth.router)
app.include_router(services.router)
app.include_router(bookings.router)
app.include_router(itineraries.router)
app.include_router(messages.router)
app.include_router(reviews.router)
app.include_router(costs.router)
app.include_router(admin.router)
app.mount("/", ws.router)
```

Cada router aporta sus endpoints. La diferencia técnica: `include_router` para rutas HTTP normales y `mount` para WebSockets (conexión en tiempo real, el chat).

Analogía

El restaurante abre y se le pegan los menús: “a la carta”, “especiales”, “menú del día”.

6.6 La ruta raíz (líneas 36–38)

```
@app.get("/")
def root():
    return {"message": "TravelHub API running"}
```

Es el “timbre de bienvenida”: al visitar `http://localhost:8000/` confirma que el backend está vivo.

6.7 Resumen del flujo completo

Cuando el backend arranca, `main.py` hace, en orden:

1. **Invita al personal** (imports).
2. **Prepara la base de datos** (crea tablas y aplica parches).
3. **Crea el cerebro** (app).
4. **Abre las puertas** (CORS).
5. **Pega los menús** (routers).
6. **Enciende el letrero** de bienvenida (raíz).

Todo esto se ejecuta con:

```
venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Resumen mental

“Toma app de main.py y ejecútalo.” Así de simple es arrancar el backend.

7. config.py — la central de configuraciones

7.1 ¿Qué es?

Es el archivo que reúne **las variables de configuración globales** del proyecto: la clave secreta, el algoritmo de los tokens, la base de datos y las claves de servicios externos. Todo el código lo lee de aquí para no tener valores escritos “a mano” en cada archivo.

7.2 Contenido completo (7 líneas)

```
import os

SECRET_KEY = os.getenv("TRAVELHUB_SECRET_KEY", "
    mi_clave_secreta_super_segura")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("
    TRAVELHUB_TOKEN_EXPIRE_MINUTES", "30"))
DATABASE_URL = os.getenv("TRAVELHUB_DATABASE_URL", "sqlite:///./
    travelhub.db")
GOOGLE_MAPS_API_KEY = os.getenv("GOOGLE_MAPS_API_KEY", "")
```

7.3 Explicación concepto por concepto

7.3.1. import os

Trae el módulo `os` de Python, que permite leer **variables de entorno** (datos que el sistema operativo le pasa al programa). Es la forma de “meter configuraciones desde fuera, sin tocar el código”.

7.3.2. `os.getenv(nombre, valor_por_defecto)` — la estrella del archivo

```
os.getenv("NOMBRE", "valor por defecto")
```

Significa: “**búscame** una variable de entorno llamada `NOMBRE`; **si existe**, úsala; **si no**, usa el valor por defecto”. Es como preguntar: “¿dejaron una nota? Si sí, la leo; si no, uso mi plan B”.

Analogía

`os.getenv` es como **revisar si dejaron una nota en la nevera** antes de salir: si la nota existe (variable de entorno), obedeces la nota; si no, usas tu plan de siempre.

7.3.3. Las variables, una por una

Variable	Variable de entorno	Para qué sirve
SECRET_KEY	TRAVELHUB_SECRET_KEY	La “firma” secreta para crear y verificar tokens JWT. ¡Nunca debe filtrarse!
ALGORITHM	— (fijo)	Algoritmo de firma del JWT: HS256 (firma simétrica con la clave secreta).
ACCESS_TOKEN_EXPIRE_MINUTES	TRAVELHUB_TOKEN_EXPIRE_MINUTES	Minutos de validez de un token (por defecto 30). <code>int()</code> lo convierte en número entero.
DATABASE_URL	TRAVELHUB_DATABASE_URL	Dónde está la base de datos. Por defecto SQLite local: <code>sqlite:///./travelhub.db</code> .
GOOGLE_MAPS_API_KEY	GOOGLE_MAPS_API_KEY	Clave de la API de Google Maps (mapas del proyecto). Vacía por defecto.

7.3.4. ¿Por qué existe el “valor por defecto”?

Para que el proyecto **funcione en tu PC sin configurar nada**. Si el ingeniero no creó la variable de entorno, el código igual corre con el valor que ya trae escrito. Eso es ideal para desarrollo local.

7.3.5. ¿Y qué pasa en producción (el servidor)?

En el servidor (Render, por ejemplo) el ingeniero **sí define las variables de entorno** (la clave real, la base de datos PostgreSQL, la clave de Maps). Así, el mismo código se comporta distinto según el entorno: **el código no cambia, cambian los valores**.

Resumen mental

`config.py` = la **central de configuraciones**. Un solo lugar para leer datos que varían entre tu PC (desarrollo) y el servidor (producción). La clave secreta nunca debe estar expuesta públicamente.

8. db/database.py — la conexión con la base de datos

8.1 ¿Qué es?

Es el archivo que **se conecta con la base de datos** y le da a toda la app los “tubos” por donde viajan los datos. En `main.py` ya lo vimos aparecer: `from app.db.database import engine, Base`. Aquí es donde nace todo eso.

8.2 Contenido completo (19 líneas)

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
from app.config import DATABASE_URL

connect_args = {}
if "sqlite" in DATABASE_URL:
    connect_args["check_same_thread"] = False

engine = create_engine(DATABASE_URL, connect_args=connect_args)
```

```

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=
    engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

8.3 Explicación concepto por concepto

8.3.1. Los imports (líneas 1–4)

- `create_engine`: crea el “tubo” que conecta Python con la base de datos.
- `declarative_base`: genera la “plantilla base” de la que saldrán las tablas (los modelos).
- `sessionmaker` y `Session`: herramientas para abrir “sesiones” de trabajo con la base.
- `DATABASE_URL`: viene de `config.py` (“dónde está la base de datos”).

8.3.2. El detalle de SQLite (líneas 6–8)

```

connect_args = {}
if "sqlite" in DATABASE_URL:
    connect_args["check_same_thread"] = False

```

SQLite es la base de datos local (el archivo `travelhub.db`) y tiene una restricción: solo permite usarla desde **un solo hilo** (“carril”) a la vez. Como FastAPI puede atender varias peticiones a la vez (varios hilos), se le dice `check_same_thread = False`: “**permite que varios carriles la usen**”.

Analogía

SQLite es una **cafetería de un solo mostrador**. Normalmente solo una persona puede pedir a la vez; con `check_same_thread = False` dejamos que **varios clientes pidan al mismo tiempo** (necesario porque la web atiende a muchos usuarios a la vez).

Nota: si la base es PostgreSQL (producción), esto no aplica, por eso el `if`.

8.3.3. engine (línea 10)

```

engine = create_engine(DATABASE_URL, connect_args=connect_args)

```

El `engine` es el “tubo maestro” de conexión. Es **único** en toda la app (se crea una sola vez) y todos los demás componentes lo comparten.

8.3.4. SessionLocal (línea 11)

```

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=
    engine)

```

Una “fábrica de sesiones”. Cada vez que necesitamos trabajar con la base, pedimos una sesión nueva a `SessionLocal`.

- `autocommit=False`: los cambios **no se guardan solos**; hay que decir `db.commit()` explícitamente (como pulsar “guardar”).
- `autoflush=False`: los cambios **no se envían solos** a la base; tú decides cuándo.
- `bind=engine`: “conéctate al tubo que creamos”.

8.3.5. Base (línea 12)

```
Base = declarative_base()
```

La “plantilla madre” de donde heredarán todos los modelos (las tablas). Cuando en `main.py` se ejecuta `Base.metadata.create_all(bind=engine)`, Python mira **todos los modelos que heredan de Base** y crea sus tablas.

8.3.6. `get_db()` (líneas 14–19) — la “fábrica con limpieza”

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Es una **función generadora** (usa `yield` en vez de `return`): - Abre una sesión. - **La presta** a la función que la pida (`yield db`). - Cuando esa función termina (pase lo que pase), **cierra la sesión** (`db.close()`).

¿Por qué importa? En casi todos los endpoints verás:

```
def mi_endpoint(db: Session = Depends(get_db)):
    ...
```

FastAPI llama a `get_db`, recibe la sesión “prestada”, la usa, y al terminar la cierra sola. **Es imposible olvidarse de cerrar la conexión** (evita fugas de memoria).

Resumen mental

`engine` = el tubo maestro. `SessionLocal` = la fábrica de sesiones. `Base` = la plantilla de las tablas. `get_db` = presta una sesión y la cierra sola.

9. `db/__init__.py` — el “carnet” de carpeta

9.1 ¿Qué es?

El archivo está **vacío** (0 líneas), pero eso no significa que no sirva.

9.2 Para qué existe

En Python, una carpeta con un archivo `__init__.py` se convierte en un “paquete” (package): un módulo que puede ser importado.

Gracias a él, el código puede escribir:


```
from app.db.database import engine, Base
```

Sin él, Python **no sabría** que db es una carpeta de código importable.

9.2.1. ¿Y si está vacío? ¿Funciona?

Sí. Que esté vacío solo significa que “no tiene comportamiento propio”; sirve únicamente como **marcador de paquete**. (También puede contener código, pero aquí no hace falta.)

Analogía

`__init__.py` es el **carnet de identidad de la carpeta**: le dice a Python “soy un paquete, puedes importarme”. Vacío o no, el carnet hace su trabajo.

Resumen mental

`__init__.py` convierte una carpeta en un **paquete importable**. Puede estar vacío; su sola presencia ya cumple su función.

10. models/ — las tablas de la base de datos

10.1 ¿Qué es models/?

Carpeta que contiene **los modelos** (las “recetas” de las tablas de la base de datos). Cada archivo define una clase que **hereda de Base** (la plantilla de `database.py`) y eso la convierte en una tabla real.

Analogía

Los modelos son **moldes para galletas**: defines una vez la forma (la clase) y la base de datos hornea todas las galletas (filas) que quieras con ese mismo molde.

Los archivos de la carpeta:

- `user.py` — usuarios (turistas, prestadores, admin).
- `service.py` — servicios turísticos (el “producto” que venden los prestadores).
- `booking.py` — reservas (un turista reserva un servicio).
- `itinerary.py` — itinerarios (el plan de viaje).
- `message.py` — mensajes del chat entre turista y prestador.
- `review.py` — reseñas que dejan los turistas.
- `__init__.py` — importa todos los modelos (lo veremos al final).

Empezamos por `user.py` porque es la base de todo: las demás tablas apuntan a usuarios.

11. models/user.py — el modelo Usuario

11.1 Contenido completo (17 líneas)

```

from sqlalchemy import Column, Integer, String, Boolean, DateTime
from sqlalchemy.sql import func
from app.db.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    role = Column(String, nullable=False) # turista, prestador, admin
    verified = Column(Boolean, default=False)
    approved = Column(Boolean, default=False)
    verification_code = Column(String, nullable=True)
    verification_code_expires = Column(DateTime(timezone=True), nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

```

11.2 Explicación concepto por concepto

11.2.1. Los imports (líneas 1–3)

- `Column, Integer, String, Boolean, DateTime`: los “tipos de dato” para definir columnas (número, texto, verdadero/falso, fecha).
- `func`: utilidades de SQL (p. ej. `func.now()` = “la hora actual del servidor”).
- `Base`: la plantilla de `database.py` que convierte la clase en tabla.

11.2.2. La clase y su nombre de tabla (líneas 5–6)

```

class User(Base):
    __tablename__ = "users"

```

- `class User(Base):` — “crea una clase llamada `User` que hereda de `Base`”. Heredar = “recibe todas las habilidades de `Base`” (en este caso, saber ser una tabla).
- `__tablename__ = "users"` — le da **nombre a la tabla en la base de datos**. Ojo: la clase se llama `User`, pero la tabla se llama `users` (en plural).

11.2.3. Las columnas (líneas 8–17)

Cada línea es una **columna de la tabla**. Se lee así: “nombre = `Column(tipo, opciones)`”.

Columna	Tipo	Qué guarda / detalles
<code>id</code>	<code>Integer</code>	Número único por usuario (la “cédula”). <code>primary_key</code> = identificador oficial; <code>index</code> = se busca rápido.
<code>name</code>	<code>String</code>	Nombre del usuario. <code>nullable=False</code> = obligatorio (no puede quedar vacío).

Columna	Tipo	Qué guarda / detalles
email	String	Correo. unique = no se repite (dos usuarios no pueden tener el mismo).
password_hash	String	La contraseña encriptada (hash). Nunca se guarda la contraseña en texto plano.
role	String	El tipo de usuario: turista , prestador o admin (ver la nota de la línea 12).
verified	Boolean	¿Verificó su correo? (sí/no, por defecto False).
approved	Boolean	¿Fue aprobado? (los prestadores necesitan aprobación del admin).
verification_code	String	El código que se envía por correo para verificar. nullable=True = puede estar vacío.
verification_code_expires	DateTime	Cuándo vence el código de verificación. timezone=True = guarda la zona horaria.
created_at	DateTime	Fecha/hora en que se creó el usuario. server_default=func.now() = lo pone la base de datos sola.

11.2.4. Los 3 niveles de “restricción” (clave para novatos)

Las opciones de cada columna son **reglas de la base de datos**:

- **nullable=False**: “esta casilla es obligatoria, no aceptes un registro sin ella”.
- **unique=True**: “este valor no puede repetirse en toda la tabla”.
- **primary_key=True**: “esta columna es la **cédula oficial** del registro, única y usada para referenciarla”.

11.2.5. La diferencia default vs server_default

- **default=False** (en **verified**): el valor lo pone **Python** al crear el registro.
- **server_default=func.now()** (en **created_at**): el valor lo pone **la base de datos** (“hora de este servidor, ahora mismo”).

11.2.6. ¿De dónde salen esas columnas extra de main.py?

Recuerda el `ALTER TABLE ... ADD COLUMN` de `main.py`: añadía `verification_code`, `verification_code_expires` y `approved`. ¡Exacto! Son estas mismas columnas, añadidas después de que la tabla ya existía. Ahora se ve la conexión.

Resumen mental

Un modelo = una tabla. Una columna = un campo con tipo y reglas. **User** es la tabla **users**, con 10 columnas que describen a cada usuario.

12. models/service.py — el modelo Servicio

12.1 Contenido completo (17 líneas)

```

from sqlalchemy import Column, Integer, String, Float, Boolean, DateTime
, ForeignKey
from sqlalchemy.sql import func
from app.db.database import Base

class Service(Base):
    __tablename__ = "services"

    id = Column(Integer, primary_key=True, index=True)
    provider_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    type = Column(String, nullable=False) # guia, hotel
    name = Column(String, nullable=False)
    description = Column(String)
    price = Column(Float, nullable=False)
    location = Column(String)
    rating = Column(Float, default=0.0)
    available = Column(Boolean, default=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())

```

12.2 ¿Qué es un Service?

Es el “producto” que venden los prestadores de servicios: una **guía turística**, un **hotel**, etc. Un servicio **pertenece a un usuario** (el prestador que lo ofrece).

Analogía

Si **users** es la lista de vendedores, **services** es la **tienda**: cada tienda tiene dueño, nombre, precio y puede estar abierta o cerrada.

12.3 Lo nuevo: la columna con ForeignKey (línea 9)

```

provider_id = Column(Integer, ForeignKey("users.id"), nullable=False)

```

Esta es la **gran novedad** frente a User:

- **provider_id**: “el id del prestador (dueño) de este servicio”.
- **ForeignKey("users.id")**: **clave foránea** — “este número debe existir en la columna **id** de la tabla **users**”.

¿**Por qué importa?** Garantiza la **integridad**: no puedes crear un servicio cuyo dueño no exista. Y permite **relacionar tablas**: dado un servicio, sabes a qué usuario pertenece.

Analogía

ForeignKey es como escribir en un formulario “**folio del cliente**”: ese folio debe existir en el registro de clientes. No puedes inventarte un folio que no existe.

12.4 Las columnas, una por una

Columna	Tipo	Qué guarda / detalles
id	Integer	Identificador único del servicio.
provider_id	Integer	El id del prestador dueño. ForeignKey hacia users.id (obligatorio).
type	String	Tipo de servicio: guia o hotel (ver nota línea 10).
name	String	Nombre del servicio (obligatorio).
description	String	Descripción. Sin nullable=False: puede ser vacío .
price	Float	Precio (número con decimales). Nuevo tipo: Float.
location	String	Ubicación. Puede ser vacío.
rating	Float	Calificación promedio (empieza en 0.0).
available	Boolean	¿Está disponible? (por defecto True).
created_at	DateTime	Fecha de creación (la pone la base de datos sola).

12.4.1. El nuevo tipo Float

Integer guarda números **enteros** (1, 50, 300). Float guarda números **con decimales** (49.99, 4.5). Ideal para precios y calificaciones.

12.4.2. El patrón de las columnas sin nullable=False

Observa: description y location **no** dicen nullable=False. Eso significa que son **opcionales**: pueden quedar vacías (NULL). En User, todas las columnas importantes eran obligatorias; aquí el servicio puede no tener descripción.

Resumen mental

ForeignKey = **apuntar a otra tabla**. Un servicio apunta a su dueño (provider_id → users.id). Así las tablas se conectan entre sí.

13. models/booking.py — el modelo Reserva

13.1 Contenido completo (14 líneas)

```
from sqlalchemy import Column, Integer, String, Float, DateTime,
    ForeignKey
from sqlalchemy.sql import func
from app.db.database import Base

class Booking(Base):
    __tablename__ = "bookings"

    id = Column(Integer, primary_key=True, index=True)
    tourist_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    service_id = Column(Integer, ForeignKey("services.id"), nullable=False)
    date = Column(DateTime, nullable=False)
    status = Column(String, default="pendiente") # pendiente,
        confirmada, cancelada
    total = Column(Float, nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

13.2 ¿Qué es una Booking?

Es el “contrato” que une a **un turista** con **un servicio** en una fecha concreta. Es la tabla que **conecta a dos tablas a la vez**.

Analogía

Es como el **ticket de compra** de una tienda: dice quién compró (`tourist_id`), qué compró (`service_id`), cuándo (`date`), cuánto pagó (`total`) y en qué estado está la compra (`status`).

13.3 Lo destacado: DOS claves foráneas (líneas 9–10)

```
tourist_id = Column(Integer, ForeignKey("users.id"), nullable=False)
service_id = Column(Integer, ForeignKey("services.id"), nullable=False)
```

Booking tiene **dos apuntadores**:

- `tourist_id` → `users.id`: “**quién** reserva” (el turista).
- `service_id` → `services.id`: “**qué** reserva” (el servicio).

Esto es lo bonito de las bases relacionales: **una reserva une a un usuario con un servicio** sin copiar sus datos (no guarda el nombre del turista ni el precio del servicio; solo apunta a ellos).

Resumen mental

Una reserva **no duplica información**: guarda los `id` que apuntan al usuario y al servicio. Si cambia el precio del servicio, todas las reservas “lo ven” actualizado.

13.4 Las columnas, una por una

Columna	Tipo	Qué guarda / detalles
<code>id</code>	<code>Integer</code>	Identificador único de la reserva.
<code>tourist_id</code>	<code>Integer</code>	Id del turista que reserva. <code>ForeignKey</code> → <code>users.id</code> (obligatorio).
<code>service_id</code>	<code>Integer</code>	Id del servicio reservado. <code>ForeignKey</code> → <code>services.id</code> (obligatorio).
<code>date</code>	<code>DateTime</code>	Fecha/hora de la reserva. Obligatorio.
<code>status</code>	<code>String</code>	Estado: pendiente , confirmada o cancelada (por defecto pendiente).
<code>total</code>	<code>Float</code>	Precio total de la reserva (con decimales). Obligatorio.
<code>created_at</code>	<code>DateTime</code>	Cuándo se creó la reserva (lo pone la base de datos).

13.5 El estado como texto (`status`) — una decisión de diseño

`status` guarda el estado como **texto** (“**pendiente**”). Podría ser un número (0, 1, 2), pero el texto es **más legible** y fácil de depurar. El “significado” de los estados está anotado en el comentario de la línea 12. El frontend usa esos textos para mostrar botones (“confirmar”, “cancelar”) según el estado.

Resumen mental

Booking = la **unificadora**: une turista + servicio + fecha + precio + estado. Con dos claves foráneas ya tenemos un minisistema de tres tablas conectadas.

14. models/itinerary.py — el modelo Itinerario

14.1 Contenido completo (12 líneas)

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey,
    JSON
from sqlalchemy.sql import func
from app.db.database import Base

class Itinerary(Base):
    __tablename__ = "itineraries"

    id = Column(Integer, primary_key=True, index=True)
    tourist_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    day = Column(Integer, nullable=False)
    route_data = Column(JSON) # Guarda puntos del mapa
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

14.2 ¿Qué es una Itinerary?

Es el **plan de viaje** de un turista: “qué hago el día X”. Por eso su tabla es simple: apunta al turista (`tourist_id`), dice el número de día (`day`) y guarda la ruta en un `route_data`.

Analogía

Es como la **página de una agenda de viaje**: “Día 1: visita estos puntos del mapa”. Cada itinerario es una página, y el turista puede tener muchas páginas (días).

14.3 Lo nuevo: el tipo JSON (línea 11)

```
route_data = Column(JSON) # Guarda puntos del mapa
```

Hasta ahora los tipos eran rígidos (texto, número, fecha). JSON es **flexible**: guarda una estructura completa (objetos, listas, coordenadas) tal cual.

En este proyecto guarda **puntos del mapa** (latitud, longitud, nombre de lugares), que vienen del trabajo con `maps_utils.py` (la API de Google Maps).

14.3.1. JSON en una frase

```
"puntos": [{"lat": -12.0464, "lng": -77.0428, "lugar": "Centro"}]
```

- JSON = *JavaScript Object Notation*: formato de texto para guardar datos estructurados (parece un diccionario de Python).
- Ventaja: guardas **toda la ruta de un día en una sola casilla**, sin necesidad de crear otra tabla con los puntos.

- La contrapartida: es difícil “buscar dentro” del JSON con SQL (no es ideal para datos que se consultan a menudo). Aquí es perfecto porque solo se lee y se muestra.

14.4 Las columnas, una por una

Columna	Tipo	Qué guarda / detalles
id	Integer	Identificador único del itinerario.
tourist_id	Integer	Id del turista dueño del plan. <code>ForeignKey</code> → <code>users.id</code> (obligatorio).
day	Integer	El número del día del viaje (1, 2, 3...). Obligatorio.
route_data	JSON	La ruta del día (puntos del mapa). Puede estar vacía.
created_at	DateTime	Cuándo se creó el itinerario (lo pone la base de datos).

14.5 El patrón que se repite

Observa que `itinerary` sigue el mismo molde: `id`, algo que apunta al usuario con `ForeignKey`, datos propios, y `created_at`. En un proyecto, la mayoría de tablas siguen este patrón.

Resumen mental

JSON = guardar **datos flexibles y estructurados** (como la ruta del mapa) en una sola columna. Simple y poderoso para este caso.

15. models/message.py — el modelo Mensaje

15.1 Contenido completo (14 líneas)

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey, Boolean
from sqlalchemy.sql import func
from app.db.database import Base

class Message(Base):
    __tablename__ = "messages"

    id = Column(Integer, primary_key=True, index=True)
    sender_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    receiver_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    booking_id = Column(Integer, ForeignKey("bookings.id"), nullable=False)
    content = Column(String, nullable=False)
    read = Column(Boolean, default=False)
    timestamp = Column(DateTime(timezone=True), server_default=func.now())
```


15.2 ¿Qué es una Message?

Es un **mensaje del chat** entre un turista y un prestador dentro de una reserva. Recuerda el archivo `ws.py`: cada vez que alguien escribe en el chat, aquí se guarda el mensaje en la base de datos.

15.3 Lo destacado: tres claves foráneas (líneas 9–11)

```
sender_id    = Column(Integer, ForeignKey("users.id"), nullable=False)
receiver_id  = Column(Integer, ForeignKey("users.id"), nullable=False)
booking_id   = Column(Integer, ForeignKey("bookings.id"), nullable=False)
```

Un mensaje necesita saber **tres cosas**:

- `sender_id` → `users.id`: **quién** lo envía.
- `receiver_id` → `users.id`: **a quién** va dirigido.
- `booking_id` → `bookings.id`: **de qué reserva** se habla.

15.3.1. ¡Detalle curioso! Dos foráneas a la misma tabla

Fíjate: `sender_id` y `receiver_id` apuntan **los dos** a `users.id`. Eso es correcto y muy común: la tabla `users` puede ser referenciada **muchas veces** por la misma tabla, siempre que los nombres de las columnas sean distintos. No hay conflicto.

Analogía

Es como un **correo postal**: `sender_id` es el remitente y `receiver_id` el destinatario. Ambos son personas (de la misma tabla “personas”), pero con roles distintos.

15.4 Las columnas, una por una

Columna	Tipo	Qué guarda / detalles
<code>id</code>	Integer	Identificador único del mensaje.
<code>sender_id</code>	Integer	Quién lo envía. <code>ForeignKey</code> → <code>users.id</code> (obligatorio).
<code>receiver_id</code>	Integer	A quién va dirigido. <code>ForeignKey</code> → <code>users.id</code> (obligatorio).
<code>booking_id</code>	Integer	Reserva de la que se habla. <code>ForeignKey</code> → <code>bookings.id</code> (obligatorio).
<code>content</code>	String	El texto del mensaje (obligatorio).
<code>read</code>	Boolean	¿Ya lo leyó el destinatario? (por defecto <code>False</code>).
<code>timestamp</code>	DateTime	Cuándo se envió (lo pone la base de datos).

15.5 Cómo se conecta con `ws.py`

En `ws.py` vimos que al recibir `send_message`, el código crea:

```
new_message = Message(
    sender_id=user_id,
    receiver_id=receiver_id,
    booking_id=booking_id,
```

```

        content=content
    )
    db.add(new_message)
    db.commit()

```

Ahora ves el modelo detrás de ese código: los datos llegan por el chat (WebSocket) y se **guardan en esta tabla**. Además, `mark_read` marca `read = True`. Todo cuadra: los nombres de las columnas son los mismos que se usan en `ws.py`.

Resumen mental

`Message` = el chat **guardado**. Tres foráneas lo ubican (quién, a quién, de qué reserva) y `read` controla el “visto”.

16. models/review.py — el modelo Reseña

16.1 Contenido completo (14 líneas)

```

from sqlalchemy import Column, Integer, String, Float, DateTime,
    ForeignKey
from sqlalchemy.sql import func
from app.db.database import Base

class Review(Base):
    __tablename__ = "reviews"

    id = Column(Integer, primary_key=True, index=True)
    booking_id = Column(Integer, ForeignKey("bookings.id"), nullable=
        False)
    tourist_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    service_id = Column(Integer, ForeignKey("services.id"), nullable=
        False)
    rating = Column(Integer, nullable=False) # 1-5
    comment = Column(String)
    created_at = Column(DateTime(timezone=True), server_default=func.now
        ())

```

16.2 ¿Qué es una Review?

Es la **valoración** que deja un turista sobre un servicio que contrató. Tiene la calificación (`rating`), un comentario opcional, y **apunta a tres tablas**: la reserva, el turista y el servicio.

16.3 Lo destacado: tres claves foráneas (líneas 9–11)

```

booking_id = Column(Integer, ForeignKey("bookings.id"), nullable=False
)
tourist_id = Column(Integer, ForeignKey("users.id"), nullable=False
)
service_id = Column(Integer, ForeignKey("services.id"), nullable=False
)

```

¿Por qué tres? Para contestar tres preguntas de una reseña:

- `booking_id`: ¿a qué reserva corresponde la reseña? (evita reseñas falsas sin haber contratado).
- `tourist_id`: **quién** la escribió.
- `service_id`: a qué servicio se refiere.

16.3.1. La relación con `service.rating`

Recuerda que en `service.py` había una columna `rating` (Float, empieza en 0.0). Esa es la **calificación promedio** del servicio. Aquí en `reviews` se guardan **cada una de las notas individuales**. El promedio se calcula a partir de todas las reseñas: cada nueva reseña “alimenta” el promedio del servicio. Una tabla sirve para calcular datos de otra.

Analogía

La reseña individual es la **nota de un examen**; el `rating` del servicio es el **promedio del curso**, que se recalcula con cada examen nuevo.

16.4 Las columnas, una por una

Columna	Tipo	Qué guarda / detalles
<code>id</code>	Integer	Identificador único de la reseña.
<code>booking_id</code>	Integer	Reserva que se reseña. <code>ForeignKey</code> → <code>bookings.id</code> (obligatorio).
<code>tourist_id</code>	Integer	Turista que la escribió. <code>ForeignKey</code> → <code>users.id</code> (obligatorio).
<code>service_id</code>	Integer	Servicio reseñado. <code>ForeignKey</code> → <code>services.id</code> (obligatorio).
<code>rating</code>	Integer	Calificación de 1 a 5 (ver nota línea 12).
<code>comment</code>	String	Comentario en texto. Opcional (sin <code>nullable=False</code>).
<code>created_at</code>	DateTime	Cuándo se escribió (lo pone la base de datos).

16.4.1. Detalle: `rating` es Integer aquí

En `service.py`, `rating` era Float (4.5, 3.8 = el promedio). Aquí, en la reseña individual, `rating` es Integer porque cada turista da una nota **entera** de 1 a 5. Mismo nombre, tipo distinto: **el promedio es decimal, cada nota es entera**.

Resumen mental

`Review` = la **nota** de un servicio. Tres foráneas la ubican (reserva + turista + servicio), y alimenta el promedio `rating` de `services`.

17. `models/__init__.py` — el pegamento de los modelos

17.1 Contenido completo (6 líneas)

```
from .user import User
from .service import Service
from .booking import Booking
```

```
from .itinerary import Itinerary
from .message import Message
from .review import Review
```

17.2 ¿Qué es?

Es un `__init__.py` **con contenido**: en lugar de estar vacío, aquí hace **dos trabajos** a la vez:

1. Convierte a `models/` en un **paquete importable** (como vimos en `db/`).
2. **Importa todos los modelos** para que sean “visibles” de un solo golpe.

17.3 La importación con punto (`from .user import User`)

El **punto** (`.`) significa “**desde la misma carpeta**”. Es decir: “desde el archivo `user` (de esta carpeta), trae la clase `User`”.

Es una importación **relativa**: no busca en todo el proyecto, solo en la carpeta actual.

17.3.1. ¿Para qué se hace esto?

Fíjate cómo usa `main.py`:

```
from app.models import User, Service, Booking, Itinerary, Message,
    Review
```

Solo importa la carpeta `app.models`, no cada archivo. Gracias a `__init__.py`, esa sola línea trae **todas las clases** de golpe. Sin él, habría que escribir seis imports uno por uno:

```
from app.models.user import User
from app.models.service import Service
...
```

17.3.2. El detalle CLAVE: ¿por qué importarlos aquí?

Recuerda `main.py`:

```
Base.metadata.create_all(bind=engine)
```

Esa línea crea las tablas **de todos los modelos que ya estén importados**. Como `models/__init__.py` importa **todos**, con importar la carpeta `app.models` una vez, **Python** ya “**conoce**” las **6 tablas** y las crea todas.

Analogía

`models/__init__.py` es como el **registro de propiedades**: cuando llega el arquitecto (`main.py`) a preguntar “¿qué edificios hay?” (`create_all`), el registro le muestra la lista completa. Si algún modelo no estuviera ahí, su tabla no se crearía.

Resumen mental

`models/__init__.py` = **el registro de los modelos**. Importar `app.models` basta para que `create_all` cree las 6 tablas.

18. Resumen de models/ — las 6 tablas

Tabla	Qué guarda	A qué apunta
users	Usuarios (turista, prestador, admin)	—
services	Servicios ofrecidos (guía, hotel)	users.id (dueño)
bookings	Reservas de un turista sobre un servicio	users.id + services.id
itineraries	Plan diario con la ruta del mapa	users.id
messages	Mensajes del chat	users.id x2 + bookings.id
reviews	Reseñas con nota de 1 a 5	bookings.id + users.id + services.id

Resumen mental

Modelos completos. Ya entendemos cómo se guarda la información de TravelHub: 6 tablas conectadas por claves foráneas. El siguiente gran bloque es `schemas/` (los “formularios” de validación que usan las rutas).

19. schemas/ — los “formularios” de validación

19.1 ¿Qué es schemas/?

Carpeta que contiene los **schemas** (esquemas) de Pydantic. Si los `models/` definen **cómo se guarda** la información en la base de datos, los `schemas/` definen **qué información entra y sale** de la API y **cómo validarla**.

Analogía

Los modelos son el **cajón del archivo** donde se guardan los papeles (base de datos). Los schemas son los **formularios de entrada**: te dicen “aquí llena nombre, aquí email”. Si llenas mal el formulario, te lo devuelven con errores antes de guardarlo.

19.2 ¿Por qué se necesitan? ¿No bastan los modelos?

Seguridad y claridad. Tres razones:

1. **El usuario no debe enviar todo.** Al registrarse, el turista envía solo `name`, `email`, `password` y `role`. No le dejas elegir su propio `id` ni `created_at` (esos los pone el sistema).
2. **Validación automática.** Si el email tiene mal formato o falta un campo, Pydantic **rechaza la petición** con un mensaje de error claro, *antes* de tocar la base de datos.
3. **El frontend sabe qué enviar.** El schema es como un “contrato”: documenta qué campos espera cada endpoint.

19.3 Pydantic: el “inspector” de FastAPI

Pydantic es la librería (la vimos en `requirements.txt`) que valida datos con sintaxis de Python moderna. Un schema se escribe así:

```
from pydantic import BaseModel, EmailStr

class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str
    role: str
```

`BaseModel` es la “plantilla de formulario”. Cada línea es un campo con su tipo (`str` = texto). Pydantic revisa que lo que llega cumpla el tipo: si pides `email: EmailStr` y llega “hola”, lo rechaza.

19.3.1. Los archivos de la carpeta

- `user.py` — formularios de usuario (registro).
- `token.py` — el “carnet” que se devuelve al hacer login.
- `service.py`, `booking.py`, `itinerary.py`, `message.py`, `review.py` — formularios para crear/actualizar cada entidad.
- `cost.py` — formulario de costos.

Empezamos por `user.py`, que conecta con el modelo `User` que ya conocemos.

20. `schemas/user.py` — el formulario de registro

20.1 Contenido completo (7 líneas)

```
from pydantic import BaseModel, EmailStr

class UserCreate(BaseModel):
    name: str
    email: EmailStr
    password: str
    role: str # turista, prestador, admin
```

20.2 Explicación concepto por concepto

20.2.1. El import (línea 1)

```
from pydantic import BaseModel, EmailStr
```

- `BaseModel`: la plantilla base de Pydantic para crear un formulario.
- `EmailStr`: un tipo especial que **solo acepta emails bien formados** (por eso en `requirements.txt` estaba `email-validator`).

20.2.2. La clase UserCreate (líneas 3–7)

El nombre dice **para qué sirve**: `User + Create` = “los datos que el cliente envía para **crear** un usuario”.

Cada línea es un campo: `nombre: tipo`.

- `name: str` — el nombre, texto.
- `email: EmailStr` — el correo, validado como correo.
- `password: str` — la contraseña, texto.
- `role: str` — el rol (turista, prestador, admin).

20.2.3. Los “poderes” que da BaseModel

Al heredar de `BaseModel`, el formulario gana herramientas automáticas:

- **Validación**: si falta un campo o el tipo no coincide, Pydantic responde 422 (“petición inválida”) con los errores.
- **Conversión**: si llega `role` como algo convertible a texto, lo pasa a `str`.
- **Serialización**: convierte el formulario a JSON fácilmente (`.model_dump()`).

20.3 Comparación: `models/User` vs `schemas/UserCreate`

Aspecto	<code>models/user.py</code>	<code>schemas/user.py</code>
Para qué	Guardar en la base de datos	Recibir/validar datos del cliente
Base	<code>Base</code> (SQLAlchemy)	<code>BaseModel</code> (Pydantic)
Campos	10 columnas (id, created_at...)	Solo 4 campos que envía el cliente
Contraseña	Guarda <code>password_hash</code>	Recibe <code>password</code> (texto plano)
Valida	Reglas de la base (nullable, unique)	Tipos (str, EmailStr)

20.3.1. El detalle de la contraseña

El formulario recibe `password` en **texto plano** (lo que el usuario escribe). El modelo guarda `password_hash` (**encriptada**). ¿Quién hace la conversión? La ruta `auth` (que veremos pronto): recibe el formulario, **encripta** la contraseña, y guarda el hash en el modelo. **El schema y el modelo nunca mezclan sus responsabilidades**.

Resumen mental

`schemas` = **formularios de entrada** (Pydantic). `models` = **cajones de guardado** (SQLAlchemy). El schema valida lo que llega; el modelo define cómo se guarda.

21. `schemas/token.py` — los formularios del login

21.1 Contenido completo (9 líneas)

```
from pydantic import BaseModel

class Token(BaseModel):
    access_token: str
    token_type: str

class LoginRequest(BaseModel):
    email: str
    password: str
```

21.2 ¿Qué contiene? Dos clases en un solo archivo

A diferencia de `user.py`, aquí hay **dos formularios** en el mismo archivo. Cada uno tiene un trabajo distinto:

21.3 LoginRequest — lo que el cliente envía

```
class LoginRequest(BaseModel):
    email: str
    password: str
```

Es el formulario de **entrada**: el cliente envía `email` y `password` cuando quiere iniciar sesión. Sin este schema, la ruta `auth` tendría que adivinar qué campos esperar.

21.4 Token — lo que el servidor devuelve

```
class Token(BaseModel):
    access_token: str
    token_type: str
```

Es el formulario de **salida**: el servidor responde con el “carnet” de acceso.

- `access_token`: el **token JWT** (texto largo encriptado).
- `token_type`: el tipo, casi siempre `"bearer"` (significa “este token autoriza al que lo lleva”).

Analogía

`LoginRequest` es la **solicitud del pase** (“me llamo X y esta es mi contraseña”). `Token` es el **pase de acceso** que te dan al entrar (“este es tu carnet”). Con él puedes pedir las cosas restringidas sin volver a mostrar tu contraseña.

21.5 Cómo funciona el login con estos dos formularios

1. El frontend envía `LoginRequest` (`email` + `password`).
2. La ruta `auth` verifica las credenciales en la base.
3. Si son correctas, crea un **token JWT** (usando `SECRET_KEY` y `ALGORITHM` de `config.py`).
4. Devuelve un `Token` con el `access_token`.
5. El frontend guarda el token y lo envía en cada petición siguiente, como prueba de “ya entré”.

21.5.1. El detalle de la validez del token

Recuerda `config.py`: `ACCESS_TOKEN_EXPIRE_MINUTES = 30`. Eso significa que el token tiene **fecha de caducidad**: después de 30 minutos deja de servir y hay que volver a iniciar sesión. Así, si el token se roba, sirve de poco.

Resumen mental

`LoginRequest` = qué envía el cliente (entrada). `Token` = qué devuelve el servidor (salida). Ambos forman el “rito” del login con JWT.

22. `schemas/service.py` — formularios de servicio

22.1 Contenido completo (17 líneas)

```
from pydantic import BaseModel
from typing import Optional

class ServiceCreate(BaseModel):
    type: str # "guia" o "hotel"
    name: str
    description: str
    price: float
    location: str

class ServiceUpdate(BaseModel):
    type: Optional[str] = None
    name: Optional[str] = None
    description: Optional[str] = None
    price: Optional[float] = None
    location: Optional[str] = None
    available: Optional[bool] = None
```

22.2 ¿Qué contiene? Dos formularios: crear y actualizar

Este archivo introduce un patrón nuevo: **dos clases para dos operaciones distintas**.

22.3 `ServiceCreate` — crear un servicio

```
class ServiceCreate(BaseModel):
    type: str
    name: str
    description: str
    price: float
    location: str
```

Es el formulario de **creación**: todos los campos son **obligatorios** (no tienen valor por defecto). El cliente que quiere publicar un servicio debe enviarlos todos.

- `type`: "guia" o "hotel" (igual que el modelo).
- `price`: `float`: nota el tipo `float` (Pydantic usa minúsculas; en SQLAlchemy era `Float`).
- **Ojo**: no incluye `provider_id`. Ese lo pone la **ruta** automáticamente con el usuario autenticado (“el que crea el servicio es su dueño”).

22.4 ServiceUpdate — actualizar un servicio

```
class ServiceUpdate(BaseModel):
    type: Optional[str] = None
    ...
    available: Optional[bool] = None
```

Es el formulario de **actualización** y es la novedad pedagógica de hoy:

- `Optional[str]` = “este campo puede venir **o no**” (puede ser texto o `None`).
- `= None` = si no se envía, por defecto vale `None` (“no se tocó”).
- Incluye `available`, que no estaba en `Create`: para activar/desactivar un servicio (abrirlo o cerrarlo) sin reescribir los demás datos.

22.4.1. ¿Por qué `Optional` es importante aquí?

Para actualizar solo una parte. Si un prestador quiere cambiar solo el precio, envía *solo price*. Con `Optional`, los demás campos son `None` y la ruta sabe que “no se deben tocar”. Si fueran obligatorios, tendría que reenviar todo el servicio.

Analogía

`ServiceCreate` es el **formulario de alta** completo (todos los datos nuevos). `ServiceUpdate` es el **formulario de cambios**: rellenas solo la casilla que quieres modificar y dejas las demás en blanco.

22.5 Los patrones de Pydantic que ya conocemos... y el nuevo

Patrón	Significado	Cuándo se usa
<code>campo: str</code>	Obligatorio	Crear (todo llega completo)
<code>campo: str = "x"</code>	Valor por defecto	Crear con valor inicial
<code>campo: Optional[str] = None</code>	Opcional	Actualizar (solo lo que cambia)

Resumen mental

`ServiceCreate` = formulario completo (todo obligatorio). `ServiceUpdate` = formulario parcial (todo opcional con `Optional`). El patrón “uno para crear, otro para actualizar” se repetirá en el resto de la carpeta.

23. schemas/booking.py — formularios de reserva

23.1 Contenido completo (9 líneas)

```
from pydantic import BaseModel
from datetime import datetime

class BookingCreate(BaseModel):
```

```

    service_id: int
    date: datetime

class BookingStatusUpdate(BaseModel):
    status: str # confirmada, cancelada

```

23.2 ¿Qué contiene? Formularios de crear y cambiar estado

23.3 BookingCreate — reservar un servicio

```

class BookingCreate(BaseModel):
    service_id: int
    date: datetime

```

El turista solo envía **dos cosas** para reservar:

- **service_id**: **qué** servicio quiere (apunta al servicio).
- **date**: **cuándo** (fecha y hora).

23.3.1. El nuevo tipo datetime

- **datetime** (de la librería `datetime` de Python): representa **fecha y hora** a la vez (ej. "2026-08-10 14:30").
- Pydantic lo **valida y lo convierte**: si llega un texto con formato de fecha, lo transforma automáticamente a un objeto `datetime` antes de guardarlo.

23.3.2. ¿Qué no envía el cliente?

Fíjate lo que falta (comparando con el modelo `Booking`):

- **tourist_id**: lo pone la **ruta** (el turista autenticado).
- **total**: lo calcula la **ruta** con el precio del servicio.
- **status**: empieza en "pendiente" solo.
- **id** y **created_at**: los pone el sistema.

El cliente no puede decidir el estado ni el total: eso lo controla el servidor. **Seguridad**.

23.4 BookingStatusUpdate — cambiar el estado

```

class BookingStatusUpdate(BaseModel):
    status: str # confirmada, cancelada

```

Es un formulario **mini**: solo un campo. Se usa cuando un prestador (o turista) **confirma o cancela** una reserva. Envía el nuevo estado y la ruta lo actualiza.

23.5 Comparación rápida con el modelo `Booking`

Campo	Modelo Booking	Quién lo llena
service_id	Sí	El cliente (BookingCreate)
date	Sí	El cliente (BookingCreate)
tourist_id	Sí	La ruta (usuario autenticado)
total	Sí	La ruta (calculado del servicio)
status	Sí	Empieza pendiente; cambia con BookingStatusUpdate
id / created_at	Sí	El sistema

Resumen mental

BookingCreate = el turista elige **qué** (service_id) y **cuándo** (date).
BookingStatusUpdate = confirma o cancela. Lo demás lo controla el servidor.

24. schemas/itinerary.py — formularios del itinerario

24.1 Contenido completo (30 líneas)

```

from pydantic import BaseModel
from typing import Optional, List, Dict, Any

class ItineraryCreate(BaseModel):
    day: int
    route_data: Dict[str, Any]

class ItineraryUpdate(BaseModel):
    route_data: Dict[str, Any]

class RoutePoint(BaseModel):
    lat: float
    lng: float
    name: str = ""
    order: int = 0

class DirectionsRequest(BaseModel):
    origin: RoutePoint
    destination: RoutePoint
    waypoints: List[RoutePoint] = []

class DirectionsResponse(BaseModel):
    total_distance_m: float
    total_distance_km: float
    total_duration_s: float
    total_duration_min: float
    total_duration_text: str
    legs: List[Dict[str, Any]]
    polyline: str
    source: str

```

24.2 ¿Qué contiene? Cinco clases (el archivo más completo)

Este archivo tiene **5 formularios**, y es el más interesante de la carpeta porque introduce **tipos nuevos** y el concepto de **formularios anidados**.

24.3 ItineraryCreate y ItineraryUpdate — el plan diario

```
class ItineraryCreate(BaseModel):
    day: int
    route_data: Dict[str, Any]

class ItineraryUpdate(BaseModel):
    route_data: Dict[str, Any]
```

- ItineraryCreate: día del viaje (`int`) + la ruta (`route_data`).
- ItineraryUpdate: solo la ruta (se actualiza el plan de ese día).
- `Dict[str, Any]`: un **diccionario** (la “bolsa” con nombre → valor). Aquí: “guardar cualquier cosa bajo etiquetas” (los puntos del mapa, como vimos en el modelo).

24.3.1. El nuevo tipo `Dict[str, Any]`

- `Dict` = diccionario: pares clave: valor (como un JSON en Python).
- `Dict[str, Any]` = “claves de texto, valores de **lo que sea**” (`Any`).
- Es **flexible**: ideal cuando no sabes exactamente la estructura que vendrá (datos del mapa).

24.4 RoutePoint — un punto de la ruta (formulario anidado)

```
class RoutePoint(BaseModel):
    lat: float
    lng: float
    name: str = ""
    order: int = 0
```

Representa **un punto geográfico**: latitud, longitud, nombre opcional y orden.

24.4.1. Los campos con valor por defecto

`name: str = ""` y `order: int = 0` tienen **valores por defecto**: si no se envían, se usan esos. Por eso son **opcionales de facto** (siempre tienen algo).

24.5 DirectionsRequest — pedir la ruta entre puntos

```
class DirectionsRequest(BaseModel):
    origin: RoutePoint
    destination: RoutePoint
    waypoints: List[RoutePoint] = []
```

Aquí está el **formulario anidado**: los campos no son tipos simples, sino **otros formularios** (`RoutePoint`).

- **origin**: el punto de **partida** (un `RoutePoint`).
- **destination**: el punto de **llegada** (un `RoutePoint`).
- **waypoints**: puntos **intermedios**. El tipo `List[RoutePoint]` = “una **lista** de puntos” (0 o más). Por defecto, lista vacía.

24.5.1. El nuevo tipo `List`

`List[X]` = “una lista de elementos de tipo `X`”. Aquí: una lista de puntos intermedios para rutas con varias paradas (“sal de A, pasa por B y C, llega a D”).

Analogía

`DirectionsRequest` es como pedir indicaciones a la app de mapas: “saliendo de **origen**, pasando por **waypoints**, hasta **destino**”. Cada punto es un formulario dentro del formulario (matrioshka).

24.6 `DirectionsResponse` — la respuesta del mapa

```
class DirectionsResponse(BaseModel):
    total_distance_m: float
    total_distance_km: float
    total_duration_s: float
    total_duration_min: float
    total_duration_text: str
    legs: List[Dict[str, Any]]
    polyline: str
    source: str
```

Es un formulario de **salida**: define la forma de la respuesta que devuelve el servidor cuando se le pide una ruta. Contiene:

- Distancia total en metros y kilómetros.
- Duración total en segundos, minutos y como texto legible.
- **legs**: las “etapas” del viaje (lista de diccionarios).
- **polyline**: el “hilo” codificado del recorrido (para dibujarlo en el mapa).
- **source**: de dónde salió la respuesta (Google Maps u otra fuente).

¿De dónde sale esta respuesta? Del trabajo con `maps_utils.py` y la API de Google Maps. Aún no lo vemos, pero ya sabes que su salida sigue esta plantilla.

24.7 Los formularios anidados (la idea clave del archivo)

Un formulario puede contener **otros formularios** como campos. Es como las muñecas rusas: `DirectionsRequest` contiene `RoutePoint`, y `RoutePoint` contiene `float` y `str`. Pydantic valida **todo el árbol** automáticamente.

Resumen mental

Este archivo muestra lo potente de Pydantic: formularios **anidados** (`DirectionsRequest` usa `RoutePoint`) y tipos **flexibles** (`Dict[str, Any]`, `List[...]`). El itinerario guarda la ruta; las direcciones la calculan con el mapa.

25. schemas/message.py — formulario del mensaje

25.1 Contenido completo (7 líneas)

```
from pydantic import BaseModel
from datetime import datetime

class MessageCreate(BaseModel):
    receiver_id: int
    booking_id: int
    content: str
```

25.2 ¿Qué contiene? Un solo formulario

Es el archivo **más corto de la carpeta**: solo MessageCreate, con 3 campos.

- **receiver_id**: a **quién** va el mensaje (el id del destinatario).
- **booking_id**: de **qué reserva** se habla.
- **content**: el **texto** del mensaje.

25.3 Lo que el cliente no envía

Recuerda el modelo Message: tenía **sender_id**, **read** y **timestamp**. Aquí no están:

- **sender_id**: lo pone la **ruta** (“el que escribe eres tú, el autenticado”).
- **read**: siempre empieza en **False** (recién enviado, aún no leído).
- **timestamp**: lo pone la base de datos.

25.4 Cómo se conecta con ws.py y el modelo Message

En **ws.py** el mensaje llega por el **WebSocket** y no usa este formulario (la validación la hace a mano con diccionarios). Pero **MessageCreate** existe para las rutas **messages** (las peticiones HTTP normales): sirve el mismo propósito, validando **receiver_id**, **booking_id** y **content** antes de crear el mensaje.

Analogía

Es como la **planilla del mensajero**: para enviar un mensaje solo llenas “destinatario”, “asunto (reserva)” y “texto”. El remitente (sender) y la fecha los pone la oficina.

Resumen mental

MessageCreate = 3 campos mínimos: **a quién**, **de qué reserva** y **el texto**. El emisor, el leído y la fecha los controla el servidor.

26. schemas/review.py — formularios de la reseña

26.1 Contenido completo (20 líneas)

```
import datetime
from typing import Optional
from pydantic import BaseModel

class ReviewCreate(BaseModel):
    booking_id: int
    rating: int # 1-5
    comment: str = ""

class ReviewResponse(BaseModel):
    id: int
    booking_id: int
    tourist_id: int
    service_id: int
    rating: int
    comment: str
    created_at: Optional[datetime.datetime] = None

    class Config:
        from_attributes = True
```

26.2 ¿Qué contiene? Un formulario de entrada y otro de salida

Este archivo introduce el **segundo gran patrón** de Pydantic: el par **Create** (entrada) y **Response** (salida). Lo vimos con `token.py` (`LoginRequest/Token`), pero aquí está completo.

26.3 ReviewCreate — lo que envía el turista

```
class ReviewCreate(BaseModel):
    booking_id: int
    rating: int # 1-5
    comment: str = ""
```

- `booking_id`: a qué reserva corresponde la reseña (obligatorio).
- `rating`: la nota de 1 a 5 (obligatorio).
- `comment: str = ""`: el comentario, con valor por defecto "" (si no se envía, se guarda vacío; no es obligatorio).

26.4 ReviewResponse — lo que devuelve el servidor

```
class ReviewResponse(BaseModel):
    id: int
    booking_id: int
    tourist_id: int
    service_id: int
    rating: int
    comment: str
    created_at: Optional[datetime.datetime] = None
```

Es la “foto completa” de una reseña que el cliente recibe como respuesta. Incluye campos que el cliente **no envió**: `id`, `tourist_id`, `service_id` y `created_at`. Son datos que ya estaban en la base y se devuelven.

26.4.1. La novedad: `class Config: from_attributes = True`

```
class Config:
    from_attributes = True
```

Esta configuración le dice a Pydantic: “**puedes construirme a partir de un objeto de la base de datos** (un modelo SQLAlchemy), no solo de un diccionario”.

Gracias a esto, la ruta puede hacer simplemente:

```
return ReviewResponse.model_validate(resena_de_la_base)
```

Pydantic “lee” los atributos del objeto de SQLAlchemy (`review.id`, `review.rating`, ...) y arma el formulario automáticamente. Sin esta línea, tendrías que convertir el modelo a mano, campo por campo.

Analogía

Es como darle al secretario **permiso de leer tu expediente**: con `from_attributes` Pydantic puede “leer” directamente los datos guardados (el objeto del modelo) y llenar el formulario de salida sin que le dicten campo por campo.

26.5 Los tres patrones de la carpeta `schemas/`

Patrón	Ejemplo	Para qué
Create / Update	<code>ServiceCreate</code> , <code>ServiceUpdate</code>	Crear (todo) o actualizar (parcial)
Entrada / Salida	<code>LoginRequest</code> / <code>Token</code>	Lo que envía el cliente / lo que responde
Create / Response	<code>ReviewCreate</code> / <code>ReviewResponse</code>	Enviar datos / devolver la “foto” completa

Resumen mental

`ReviewCreate` = lo que el turista envía (3 campos). `ReviewResponse` = lo que el servidor devuelve (la reseña completa). `from_attributes` permite armar la respuesta directamente desde el objeto de la base.

27. `schemas/cost.py` — el formulario de costos

27.1 Contenido completo (5 líneas)

```
from pydantic import BaseModel
from typing import List

class CostCalculateRequest(BaseModel):
    booking_ids: List[int]
```

27.2 ¿Qué contiene? Un solo formulario, un solo campo

Es el archivo **más simple de la carpeta**: una clase con un único campo.

- `booking_ids`: una **lista de ids de reservas** (`List[int]`).
- Recuerda el tipo `List` de `itinerary.py`: aquí es “una lista de números enteros”.

27.3 ¿Para qué sirve?

Para **calcular el costo total de varias reservas a la vez**. El turista envía los ids de las reservas que quiere “pagar” y el servidor (la ruta `costs`) suma sus totales y devuelve el costo total.

Analogía

Es como pasar al cajero el **montón de tickets** de compra: le entregas los números de los tickets (`booking_ids`) y él te dice cuánto pagas en total.

27.4 ¿Por qué es una lista y no un número solo?

Porque el usuario puede querer liquidar **varias reservas juntas** (una sola “cuenta”). Con `List[int]` se aceptan de una a muchas reservas en la misma petición, en vez de llamar al endpoint una vez por cada reserva.

27.4.1. ¿Y por qué no se define la respuesta aquí?

La respuesta (el total calculado) es un dato simple que la ruta `costs` devuelve directamente como JSON (`{"total": 150.5}`), sin necesidad de un formulario. Este archivo solo define la **entrada**.

Resumen mental

`CostCalculateRequest` = una lista de `booking_ids`. Envías los tickets y el servidor calcula el total. Cierra la carpeta `schemas/` con un ejemplo sencillo de entrada.

28. Resumen de schemas/ — formularios de la API

Archivo	Contiene	Tipo de formulario
<code>user.py</code>	<code>UserCreate</code>	Entrada (registro)
<code>token.py</code>	<code>LoginRequest</code> , <code>Token</code>	Entrada / salida (login)
<code>service.py</code>	<code>ServiceCreate</code> , <code>ServiceUpdate</code>	Crear / actualizar
<code>booking.py</code>	<code>BookingCreate</code> , <code>BookingStatusUpdate</code>	Crear / cambiar estado
<code>itinerary.py</code>	5 clases (<code>plan</code> + direcciones)	Crear / rutas de mapa
<code>message.py</code>	<code>MessageCreate</code>	Entrada (chat)
<code>review.py</code>	<code>ReviewCreate</code> , <code>ReviewResponse</code>	Entrada / salida
<code>cost.py</code>	<code>CostCalculateRequest</code>	Entrada (cálculo)

Resumen mental

Schemas completos. Los formularios definen qué entra y qué sale de la API. Con esto, la siguiente gran pieza es **routes/**: las rutas que RECIBEN estos formularios, hablan con la base de datos y devuelven las respuestas. ¡Es la parte que hace que todo suceda!

29. routes/ — los “menús” de la API

29.1 ¿Qué es routes/?

Carpeta con **los endpoints** (rutas) de la API: las direcciones URL que el frontend llama. Cada archivo agrupa las rutas de un tema:

Archivo	Tema
auth.py	Registro, verificación de email, login (entrar y salir)
services.py	Publicar, listar y buscar servicios turísticos
bookings.py	Crear y gestionar reservas
itineraries.py	Planes de viaje y rutas con el mapa
messages.py	Mensajes (versión HTTP del chat)
reviews.py	Reseñas y calificaciones
costs.py	Cálculo de costos de varias reservas
admin.py	Acciones de administrador (aprobar prestadores)
ws.py	Chat en tiempo real con WebSockets

29.2 ¿Cómo se ve una ruta?

Una ruta es una función con un **decorador** que indica método y dirección:

```
@router.post("/register")
def register(user: UserCreate, db: Session = Depends(get_db)):
    ...
```

- `@router.post("/register")`: “cuando alguien haga POST a `/register`, ejecuta esta función”.
- `user: UserCreate`: FastAPI toma el cuerpo de la petición y lo **valida** con el schema (lo que aprendimos en `schemas/`).
- `db = Depends(get_db)`: FastAPI abre una **sesión de base de datos**, se la presta a la función y la cierra al terminar (lo que aprendimos en `database.py`).

Empezamos por `auth.py`: la “puerta de entrada” de la app.

30. routes/auth.py — la puerta de entrada (181 líneas)

30.1 ¿Qué hace este archivo?

Gestiona **todo el ciclo de identidad** del usuario:

1. **Registrarse** (`/register`).

2. **Verificar el email** con un código (/verify y /resend-code).
3. **Iniciar sesión** (/login).
4. **Entrar con Google** (/google).
5. **Ver mi perfil** (/me).
6. Además, define `get_current_user`: la “máquina de revisar el carnet” que usan **todos los demás archivos** para saber quién está haciendo la petición.

30.2 Los imports (líneas 1–16)

- `jwt`: crear y verificar los tokens JWT (la librería `pyjwt`).
- `hashlib` y `os`: encriptar contraseñas y generar valores aleatorios.
- `datetime`, `timedelta`: fechas y tiempos de expiración.
- `Depends`, `APIRouter`, `HTTPException`: herramientas de FastAPI.
- `OAuth2PasswordBearer`: el “estándar” para recibir tokens en peticiones.
- `get_db`, `User`, los `schemas`, y `config.py` (clave y algoritmo).
- `email_utils`: funciones de `email_utils.py` (aún no visto): generar el código de verificación y enviar el correo.

30.3 La configuración (líneas 18–19)

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
router = APIRouter(prefix="/auth", tags=["auth"])
```

- `oauth2_scheme`: le dice a FastAPI “**cómo recibir el token**” (normalmente en un encabezado `Authorization`). Es la puerta que revisa si la petición trae carnet.
- `prefix="/auth"`: todas las rutas de aquí empiezan con `/auth`. Por eso se escribe `/register` y en realidad es `/auth/register`.
- `tags=["auth"]`: solo organiza la documentación automática de FastAPI.

30.4 Los mini-formularios locales (líneas 21–29)

```
class VerifyRequest(BaseModel):
    email: str
    code: str

class ResendRequest(BaseModel):
    email: str

class GoogleAuthRequest(BaseModel):
    id_token: str
```

Son schemas pequeños definidos **aquí mismo** (no en `schemas/`) porque son muy específicos de estas rutas:

- `VerifyRequest`: email + código (para verificar).
- `ResendRequest`: solo email (para reenviar el código).
- `GoogleAuthRequest`: el `id_token` que da Google.

30.5 Las funciones de ayuda (líneas 31–44)

30.5.1. Encriptar contraseñas: `hash_password`

```
def hash_password(password: str) -> str:
    salt = os.urandom(32).hex()
    hash_obj = hashlib.sha256((salt + password).encode())
    return f"{salt}:{hash_obj.hexdigest()}"
```

1. `salt` = una **cadena aleatoria** única (32 bytes aleatorios).
2. Mezcla el `salt` con la contraseña y los encripta juntos con **SHA-256** (una “picadora” matemática irreversible).
3. Guarda `salt:hash` juntos.

30.5.2. ¿Por qué el “salt”?

Si dos usuarios usan la misma contraseña, sus hash serían idénticos (una pista para hackers). Con un `salt` **aleatorio** por usuario, aunque dos contraseñas sean iguales, los hash son **distintos**. Además, sin el `salt`, los hackers podrían usar tablas ya calculadas (diccionarios) para adivinar contraseñas.

Analogía

El hash es como **triturar carne**: se puede hacer pero no se puede volver a armar. El `salt` es **una especia distinta en cada triturada**: aunque la carne sea igual, la mezcla final siempre es diferente.

30.5.3. Verificar contraseñas: `verify_password`

```
def verify_password(password: str, hashed: str) -> bool:
    salt, hash_value = hashed.split(":")
    return hashlib.sha256((salt + password).encode()).hexdigest() ==
        hash_value
```

Al hacer login, el usuario escribe su contraseña. Para comprobar: se **separa el salt** guardado, se vuelve a triturar `salt + contraseña`, y se compara con el hash guardado. Si coinciden, la contraseña es correcta. **La contraseña original nunca se guardó ni se compara.**

30.5.4. Crear el token: `create_access_token`

```
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=
        ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

1. Copia los datos que se quieren meter dentro del token (ej. email y rol).
2. Calcula la **fecha de vencimiento**: ahora + 30 minutos (de config.py).
3. Añade "exp" (expiration = vencimiento) a los datos.
4. Los **firma** con la **SECRET_KEY** y el **ALGORITHM**.

El JWT es como un **carnet firmado**: el contenido es legible, pero nadie puede cambiarlo sin la clave secreta (la firma dejaría de coincidir).

30.6 El registro: POST /register (líneas 46–76)

```
@router.post("/register")
def register(user: UserCreate, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == user.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="Email ya registrado")
    ...
```

Paso a paso:

1. Busca en la base si **ya existe** un usuario con ese email. Si sí, responde **400** (“Email ya registrado”).
2. **Encripta** la contraseña con `hash_password`.
3. Genera un **código de verificación** y su fecha de vencimiento (10 minutos).
4. Crea el **User** en memoria con: `verified=False`, `approved=True` si es turista / `False` si es prestador (¡recuerda la aprobación del admin!).
5. Lo **guarda** en la base (`db.add + db.commit`).
6. **Envía el correo** con el código (`send_verification_email`).
7. Devuelve un mensaje y (¡ojo!) también el **code** en la respuesta.

HTTPException: la forma de FastAPI de responder con un error. `status_code=400` significa “petición mala”. Los códigos 4xx son errores del cliente (algo mal en lo que envía).

30.6.1. Los códigos HTTP más usados en este proyecto

Código	Significado
200	Todo bien (éxito).
400	Error del cliente (p. ej. email ya registrado, código incorrecto).
401	No autorizado (p. ej. credenciales incorrectas, token inválido/expirado).
403	Prohibido (no tienes permiso aunque estés identificado).
404	No encontrado (el recurso no existe).
502	Error del servidor al hablar con otro servicio (p. ej. Google).

30.7 Verificar email: POST /verify (líneas 78–99)

```
@router.post("/verify")
def verify_email(body: VerifyRequest, db: Session = Depends(get_db)):
    db_user = db.query(User).filter(User.email == body.email).first()
    ...
    if db_user.verification_code != body.code:
        raise HTTPException(status_code=400, detail="Código incorrecto")
    if db_user.verification_code_expires < datetime.utcnow():
        raise HTTPException(status_code=400, detail="Código expirado")
    db_user.verified = True
    ...
    return {"message": ..., "access_token": token, "token_type": "bearer"}
    }
```

Paso a paso:

1. Busca al usuario por email. Si no existe, **404**.
2. Si ya está verificado, **400**.
3. Compara el código enviado con el guardado. Si no coincide, **400**.
4. Comprueba si el código **venció** (10 min). Si venció, **400**.
5. Marca **verified = True** y **limpia** el código (ya no se necesita).
6. **Crea un token** de acceso (¡verificar también inicia sesión!) y lo devuelve.

30.8 Reenviar código: POST /resend-code (líneas 101–116)

Es casi igual a **verify**, pero sin comprobar el código: genera uno **nuevo**, lo guarda (con 10 minutos nuevos) y lo **reenvía por correo**. Para cuando el usuario perdió el código anterior o se le venció.

30.9 Iniciar sesión: POST /login (líneas 118–128)

```
@router.post("/login", response_model=Token)
def login(user: LoginRequest, db: Session = Depends(get_db)):
    db_user = db.query(User).filter(User.email == user.email).first()
    if not db_user:
        raise HTTPException(status_code=401, detail="Credenciales
            incorrectas")
    if not verify_password(user.password, db_user.password_hash):
        raise HTTPException(status_code=401, detail="Credenciales
            incorrectas")
    token = create_access_token({"sub": db_user.email, "role": db_user.
        role})
    return {"access_token": token, "token_type": "bearer"}
```

Paso a paso:

1. Recibe **LoginRequest** (email + password) — el schema de **token.py**.
2. Busca al usuario por email. Si no existe, **401** (no digas si el email o la contraseña fallaron: eso protege a los usuarios).

3. Verifica la contraseña con `verify_password`. Si falla, **401**.
4. **Crea el token** con el email (`sub`) y el rol dentro.
5. Devuelve Token (el `access_token` + tipo `bearer`).

¿Por qué "sub"? Es un campo estándar de los JWT (`subject = sujeto`): identifica a quién pertenece el token. Aquí guarda el email.

30.10 Entrar con Google: POST `/google` (líneas 130–160)

```
@router.post("/google")
def google_auth(body: GoogleAuthRequest, db: Session = Depends(get_db)):
    try:
        import requests
        resp = requests.get(f"https://oauth2.googleapis.com/tokeninfo?id_token={body.id_token}")
        ...
        google_data = resp.json()
        email = google_data.get("email")
        ...
        db_user = db.query(User).filter(User.email == email).first()
        if not db_user:
            db_user = User(name=name, email=email,
                           password_hash=hash_password(os.urandom(32).hex()),
                           role="turista", verified=True)
            db.add(db_user); db.commit(); db.refresh(db_user)
        token = create_access_token({"sub": db_user.email, "role": db_user.role})
        return {"access_token": token, "token_type": "bearer", "user": {...}}
    except requests.RequestException:
        raise HTTPException(status_code=502, detail="Error al verificar token con Google")
```

Paso a paso:

1. Recibe el `id_token` (el “pase” que da Google al frontend).
2. Se lo **envía a Google** para verificarlo (no confiamos a ciegas).
3. De la respuesta saca `email` y `name`.
4. Si el email **no existe** en la base, **crea el usuario** automáticamente (rol `turista`, ya verificado; la contraseña es un valor aleatorio inútil porque el usuario entra solo con Google).
5. Crea su token de acceso y lo devuelve con los datos del usuario.
6. Si Google no responde (`requests.RequestException`), **502**.

30.11 La “máquina del carnet”: `get_current_user` (líneas 162–177)

```
def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email = payload.get("sub")
```



```

...
user = db.query(User).filter(User.email == email).first()
...
return {"id": user.id, "email": user.email, "role": user.role, "
        approved": user.approved}
except jwt.ExpiredSignatureError:
    raise HTTPException(status_code=401, detail="Token expirado")
except jwt.InvalidTokenError:
    raise HTTPException(status_code=401, detail="Token inválido")

```

¿Qué es? Una **dependencia**: una función que FastAPI ejecuta automáticamente para preparar algo antes de llegar a la ruta. Su trabajo: **leer el token de la petición y responder “¿quién eres?”**.

Paso a paso:

1. `Depends(oauth2_scheme)`: extrae el token del encabezado `Authorization`.
2. `jwt.decode`: abre y verifica el token con la clave secreta.
3. Saca el email (`sub`).
4. Busca al usuario en la base y devuelve sus datos (id, email, rol, aprobación).
5. Si el token venció (`ExpiredSignatureError`) o no es válido (`InvalidTokenError`), responde **401**.

30.11.1. ¿Cómo se usa en las demás rutas?

Cualquier endpoint que necesite “quién eres” escribe:

```

def mi_endpoint(current_user = Depends(get_current_user), ...):
    ...

```

FastAPI llama a `get_current_user` primero; si el token no es válido, **ni siquiera ejecuta el endpoint**. Si es válido, le pasa los datos del usuario. Así, **todos los archivos de routes** usan esta misma función para proteger sus rutas.

30.12 Mi perfil: GET `/me` (líneas 179–181)

```

@router.get("/me")
def get_me(current_user = Depends(get_current_user)):
    return current_user

```

Es un endpoint **protegido**: solo funciona si llevas un token válido. Devuelve tus propios datos (id, email, rol, approved). Muy útil para que el frontend sepa “quién soy” cuando la app arranca.

30.13 Resumen del archivo

Resumen mental

auth.py = la **puerta de entrada**:

- Encripta contraseñas (hash + salt).
- Crea y verifica tokens JWT (firmados con la clave secreta).
- Endpoints: /register, /verify, /resend-code, /login, /google, /me.
- Define `get_current_user`, el “revisa carnet” que usan todas las rutas.

31. routes/services.py — el CRUD de servicios (93 líneas)

31.1 ¿Qué hace este archivo?

Gestiona los **servicios turísticos**: crearlos, listarlos, buscarlos, editarlos y borrarlos. Es el ejemplo **perfecto de CRUD**:

- Create (crear): POST /services
- Read (leer): GET /services y GET /services/{id}
- Update (actualizar): PUT /services/{id}
- Delete (borrar): DELETE /services/{id}

31.2 La configuración (línea 8)

```
router = APIRouter(prefix="/services", tags=["services"])
```

Igual que en auth.py: todas las rutas de aquí empiezan con /services.

31.3 Crear servicio: POST /services (líneas 10–29)

```
@router.post("")
def create_service(service: ServiceCreate, db: Session = Depends(get_db)
, current_user = Depends(get_current_user)):
    if current_user["role"] != "prestador":
        raise HTTPException(status_code=403, detail="Solo prestadores
            pueden crear servicios")
    if not current_user.get("approved"):
        raise HTTPException(status_code=403, detail="Tu cuenta de
            prestador está pendiente de aprobación...")
    new_service = Service(
        provider_id=current_user["id"],
        type=service.type, name=service.name,
        description=service.description,
        price=service.price, location=service.location
    )
    db.add(new_service)
    db.commit()
    db.refresh(new_service)
    return new_service
```

La novedad: el doble control de permisos. La función pide `current_user = Depends(get_current_user)` — la “máquina del carnet” de `auth.py`. Con los datos del usuario en mano, hace **dos comprobaciones**:

1. `current_user["role"] != "prestador"` → si no eres prestador, **403**.
2. `not current_user.get("approved")` → si eres prestador pero aún no te aprobó el admin, **403**.

Luego crea el `Service` con una pista importante: `provider_id=current_user["id"]`. **El dueño no lo elige el cliente:** el sistema toma tu id del token. (Recuerda: en `schemas/service.py` el `provider_id` no estaba en el formulario. ¡Aquí se explica el por-qué!)

`db.refresh(new_service)`: recarga el objeto desde la base para tener los valores que la base asignó (como el id autogenerado).

31.4 Listar y buscar: GET /services (líneas 31–56)

```
@router.get("/")
def get_services(
    db: Session = Depends(get_db),
    type: str = Query(None, description="Filtrar por tipo: guia, hotel"),
    location: str = Query(None, description="Filtrar por ubicación"),
    min_price: float = Query(None, ...),
    max_price: float = Query(None, ...),
    min_rating: float = Query(None, ...),
    available: bool = Query(None, ...)
):
    query = db.query(Service)
    if type:
        query = query.filter(Service.type == type)
    if location:
        query = query.filter(Service.location.ilike(f"%{location}%"))
    if min_price is not None:
        query = query.filter(Service.price >= min_price)
    ...
    return query.all()
```

La novedad: los parámetros Query. Estos son **filtros opcionales** que el frontend manda en la URL. Por ejemplo:

```
GET /services?type=hotel&min_price=50&location=Lima
```

`Query(None, ...)` significa: “parámetro **opcional**; si no viene, vale `None`”.

El “filtro inteligente”: el código empieza con `query = db.query(Service)` (todos los servicios) y va **afinando con cada filtro** que venga:

- `Service.type == type`: tipo exacto.
- `Service.location.ilike("%ubicacion%")`: busca que la ubicación **contenga** ese texto. `ilike` = *case-insensitive* (no distingue mayúsculas/minúsculas) + **búsqueda parcial** gracias a los `%...%`.
- `Service.price >= min_price`: precio mínimo.

- `Service.price <= max_price`: precio máximo.
- `Service.rating >= min_rating`: calificación mínima.
- `Service.available == available`: si está disponible.

`query.all()` al final ejecuta y devuelve **todos los que pasaron los filtros**.

Analogía

Es como el **filtro de una tienda online**: vas marcando “tipo, precio máximo, ubicación” y la lista se va achicando. Cada `filter` agrega una condición.

31.5 Ver un servicio: GET `/services/{id}` (líneas 58–63)

```
@router.get("/{service_id}")
def get_service(service_id: int, db: Session = Depends(get_db)):
    service = db.query(Service).filter(Service.id == service_id).first()
    if not service:
        raise HTTPException(status_code=404, detail="Servicio no encontrado")
    return service
```

La novedad: el `{service_id}` en la URL (parámetro de ruta). Cuando el frontend pide `GET /services/5`, FastAPI captura el 5 y lo pasa como `service_id`. Si no existe ese servicio, responde **404**.

31.6 Actualizar: PUT `/services/{id}` (líneas 65–80)

```
@router.put("/{service_id}")
def update_service(service_id: int, service_data: ServiceUpdate, db: Session = Depends(get_db), current_user = Depends(get_current_user)):
    service = db.query(Service).filter(Service.id == service_id).first()
    if not service:
        raise HTTPException(status_code=404, detail="Servicio no encontrado")
    if service.provider_id != current_user["id"]:
        raise HTTPException(status_code=403, detail="Solo el prestador puede modificar su servicio")
    update_data = service_data.dict(exclude_unset=True)
    for key, value in update_data.items():
        setattr(service, key, value)
    db.commit()
    db.refresh(service)
    return service
```

Paso a paso:

1. Busca el servicio; si no existe, **404**.
2. Comprueba que **tú seas el dueño**: `service.provider_id != current_user["id"]` → **403**. (¡Otro uso de la máquina del carnet!)
3. `service_data.dict(exclude_unset=True)`: convierte el formulario `ServiceUpdate` a diccionario, pero **solo con los campos que el cliente envió**. Los que no llegaron, **no se tocan** (¡la magia del `Optional` que vimos en schemas!).

4. `setattr(service, key, value)`: por cada campo, lo **asigna** al objeto del modelo.
5. `db.commit()`: guarda los cambios.

Analogía

`exclude_unset=True` es como decirle al empleado: “solo cambia lo que vienen a cambiar”. Si el cliente solo envió `price`, solo se modifica `price`; el nombre y la ubicación quedan intactos.

31.7 Borrar: DELETE /services/{id} (líneas 82–93)

```
@router.delete("/{service_id}")
def delete_service(service_id: int, db: Session = Depends(get_db),
    current_user = Depends(get_current_user)):
    service = db.query(Service).filter(Service.id == service_id).first()
    if not service:
        raise HTTPException(status_code=404, detail="Servicio no
            encontrado")
    if service.provider_id != current_user["id"]:
        raise HTTPException(status_code=403, detail="Solo el prestador
            puede eliminar su servicio")
    db.delete(service)
    db.commit()
    return {"message": "Servicio eliminado"}
```

Misma estructura: buscar (404 si no existe), verificar dueño (403 si no eres tú), `db.delete` + `db.commit` y mensaje de confirmación.

31.8 Los métodos HTTP que ya conocemos

Método	Acción	Analogía
POST	Crear	Añadir un registro nuevo
GET	Leer	Consultar sin cambiar nada
PUT	Actualizar	Modificar uno existente
DELETE	Borrar	Eliminar uno existente

Resumen mental

`services.py` = **CRUD completo** + **permisos**. Aprende este patrón: buscar, comprobar permisos, actuar, guardar. Es el molde que usan `bookings.py`, `itineraries.py`, `messages.py` y `reviews.py`.

32. routes/bookings.py — las reservas (77 líneas)

32.1 ¿Qué hace este archivo?

Gestiona las **reservas**: crear, listar las propias, ver una y cambiar su estado. Es el ejemplo perfecto de **permisos por rol**, porque aquí conviven turistas y prestadores.

32.2 Crear reserva: POST /bookings (líneas 11–32)

```
@router.post("")
def create_booking(booking: BookingCreate, db: Session = Depends(get_db)
, current_user = Depends(get_current_user)):
    if current_user["role"] != "turista":
        raise HTTPException(status_code=403, detail="Solo turistas
            pueden reservar")
    service = db.query(Service).filter(Service.id == booking.service_id)
        .first()
    if not service:
        raise HTTPException(status_code=404, detail="Servicio no
            encontrado")
    if not service.available:
        raise HTTPException(status_code=400, detail="El servicio no está
            disponible")
    new_booking = Booking(
        tourist_id=current_user["id"],
        service_id=booking.service_id,
        date=booking.date,
        total=service.price
    )
    db.add(new_booking)
    db.commit()
    db.refresh(new_booking)
    return new_booking
```

Paso a paso:

1. Solo turistas pueden reservar (rol $\neq$ turista $\rightarrow$ 403).
2. Busca el **servicio** que se quiere reservar. Si no existe, 404.
3. Comprueba que el servicio esté **disponible**. Si no, 400.
4. Crea la reserva con:
 - tourist_id=current_user["id"]: el turista eres tú (del token).
 - service_id y date: del formulario BookingCreate.
 - total=service.price: el **precio lo toma del servicio**. ¡El cliente no escribe el total! (Recuerda el schema BookingCreate: solo enviaba service_id y date. Aquí se ve el porqué: **el servidor calcula el precio**.)
5. Guarda, refresca y devuelve.

32.3 Listar reservas: GET /bookings (líneas 34–42)

```
@router.get("")
def get_bookings(db: Session = Depends(get_db), current_user = Depends(
    get_current_user)):
    if current_user["role"] == "turista":
        return db.query(Booking).filter(Booking.tourist_id ==
            current_user["id"]).all()
    elif current_user["role"] == "prestador":
        services = db.query(Service).filter(Service.provider_id ==
            current_user["id"]).all()
```

```

service_ids = [s.id for s in services]
return db.query(Booking).filter(Booking.service_id.in_(
    service_ids)).all()
return []

```

La idea: cada uno ve **SOLO** lo suyo. La respuesta depende del rol:

- **Turista:** ve las reservas donde él es el `tourist_id`.
- **Prestador:** primero busca **sus servicios** (los que él creó), junta sus ids en una lista, y luego ve las reservas de **esos servicios**. No puede ver reservas de servicios ajenos.
- **Otro rol:** lista vacía.

32.3.1. La novedad: `Booking.service_id.in_(service_ids)`

`.in_(lista)` es “que el valor esté dentro de la lista”. En español: “dame las reservas cuyo `service_id` esté entre [10, 15, 22, ...]”. Equivale a `WHERE service_id IN (10, 15, 22, ...)` en SQL.

Analogía

Es como preguntar en la oficina: “¿hay algún paquete para las oficinas 10, 15 o 22?” (en vez de preguntar una por una).

32.4 Ver una reserva: `GET /bookings/{id}` (líneas 44–54)

```

@router.get("/{booking_id}")
def get_booking(booking_id: int, db: Session = Depends(get_db),
    current_user = Depends(get_current_user)):
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Reserva no
            encontrada")
    service = db.query(Service).filter(Service.id == booking.service_id)
        .first()
    if current_user["id"] != booking.tourist_id and current_user["id"]
        != service.provider_id:
        raise HTTPException(status_code=403, detail="No autorizado")
    return booking

```

El control de acceso “o el turista o el prestador”. Para ver una reserva, debes ser el turista que la hizo O el prestador del servicio:

- `current_user["id"] != booking.tourist_id` → no eres el turista.
- `and current_user["id"] != service.provider_id` → y tampoco eres el prestador.
- Si ambas se cumplen, **403**.

Para saber el prestador, primero busca el servicio de esa reserva. Es otra muestra de **cómo se conectan las tablas** (`booking` → `service` → `provider`).

32.5 Cambiar estado: PUT /bookings/{id}/status (líneas 56–77)

```
@router.put("/{booking_id}/status")
def update_booking_status(booking_id: int, status_data:
    BookingStatusUpdate, db: Session = Depends(get_db), current_user =
    Depends(get_current_user)):
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Reserva no
            encontrada")
    service = db.query(Service).filter(Service.id == booking.service_id)
        .first()

    if current_user["role"] == "prestador" and current_user["id"] ==
        service.provider_id:
        if status_data.status not in ["confirmada", "cancelada"]:
            raise HTTPException(status_code=400, detail="Estado inválido
                ")
    elif current_user["role"] == "turista" and current_user["id"] ==
        booking.tourist_id:
        if status_data.status != "cancelada":
            raise HTTPException(status_code=400, detail="El turista solo
                puede cancelar reservas")
    else:
        raise HTTPException(status_code=403, detail="No autorizado")

    booking.status = status_data.status
    db.commit()
    db.refresh(booking)
    return booking
```

¿Qué puede hacer cada quién? ¡Reglas de negocio distintas por rol!

Rol	Puede poner	No puede poner
Prestador (dueño del servicio)	confirmada o cancelada	Cualquier otro estado (400)
Turista (dueño de la reserva)	Solo cancelada	Confirmar (400)
Cualquier otro	Nada	403

Paso a paso:

1. Busca la reserva; si no existe, **404**.
2. Busca el servicio para conocer al prestador.
3. **if / elif / else** según el rol:
 - Si eres el prestador dueño: solo acepta **confirmada/cancelada**.
 - Si eres el turista dueño: solo **cancelada** (no puede confirmar su propia reserva).
 - Si no eres ninguno de los dos: **403**.
4. Si pasó las reglas, asigna el estado, guarda y devuelve.

¿Por qué el turista solo puede cancelar? Es una **regla de negocio**: confirmar una reserva lo decide el prestador (acepta al turista). El turista solo tiene poder para **retirarse** (cancelar). Así el sistema protege el flujo del negocio, no solo la seguridad.

Resumen mental

`bookings.py` = **permisos por rol + conexión entre tablas**. Cada quien ve solo lo suyo; cada rol solo puede hacer ciertas acciones (turista cancela, prestador confirma).

33. `routes/itineraries.py` — los planes de viaje (131 líneas)

33.1 ¿Qué hace este archivo?

Gestiona los **itinerarios** del turista: CRUD del plan, **agregar/quitar reservas** al plan, listar las reservas del plan (junto con datos del servicio) y **calcular rutas** con Google Maps.

Es el archivo que más muestra el poder de la **columna JSON** del modelo `Itinerary`.

33.2 CRUD básico (líneas 13–60)

```
@router.post("")
def create_itinerary(itinerary: ItineraryCreate, db: Session = Depends(
    get_db), current_user = Depends(get_current_user)):
    if current_user["role"] != "turista":
        raise HTTPException(status_code=403, detail="Solo turistas
            pueden crear itinerarios")
    new_itinerary = Itinerary(
        tourist_id=current_user["id"],
        day=itinerary.day,
        route_data=itinerary.route_data
    )
    db.add(new_itinerary); db.commit(); db.refresh(new_itinerary)
    return new_itinerary
```

Es el mismo patrón que ya conocemos, pero **solo para turistas**:

- `POST /itineraries`: crear (rol turista obligatorio).
- `GET /itineraries`: listar **mis** itinerarios (`tourist_id == yo`).
- `GET /itineraries/{id}`: ver uno (si es tuyo, si no **403**).
- `PUT /itineraries/{id}`: actualizar la `route_data`.
- `DELETE /itineraries/{id}`: borrar (si es tuyo).

No hay nada nuevo aquí: es el molde “buscar, verificar dueño, actuar” que ya dominamos. Lo interesante viene en las rutas que siguen.

33.3 Agregar una reserva al plan (líneas 62–79)

```
@router.post("/{itinerary_id}/add-booking/{booking_id}")
def add_booking_to_itinerary(itinerary_id: int, booking_id: int, db:
    Session = Depends(get_db), current_user = Depends(get_current_user)):
    itinerary = db.query(Itinerary).filter(Itinerary.id == itinerary_id)
    .first()
```

```

if not itinerary:
    raise HTTPException(status_code=404, detail="Itinerario no
    encontrado")
if itinerary.tourist_id != current_user["id"]:
    raise HTTPException(status_code=403, detail="No autorizado")
booking = db.query(Booking).filter(Booking.id == booking_id).first()
if not booking:
    raise HTTPException(status_code=404, detail="Reserva no
    encontrada")
if booking.tourist_id != current_user["id"]:
    raise HTTPException(status_code=403, detail="No autorizado")
if "booking_ids" not in itinerary.route_data:
    itinerary.route_data["booking_ids"] = []
if booking_id not in itinerary.route_data["booking_ids"]:
    itinerary.route_data["booking_ids"].append(booking_id)
db.commit()
return {"message": "Reserva agregada al itinerario"}

```

La novedad: dos ids en la URL ({itinerary_id}/add-booking/{booking_id}). Una URL con dos parámetros de ruta.

Los controles (los más estrictos del archivo):

1. El itinerario debe existir (404) y ser **tuyo** (403).
2. La reserva debe existir (404) y ser **tuya** (403).
3. Solo así puedes **agregar tu reserva a tu itinerario**.

La magia del JSON (líneas 74–77): aquí se manipula la columna `route_data` como un diccionario de Python:

1. Si `route_data` no tiene la clave "booking_ids", la crea con una lista vacía.
2. Si la reserva **no está** en la lista, la **agrega** (sin duplicados).
3. `db.commit()` guarda el JSON modificado.

Analogía

Es como escribir en tu **agenda de viaje**: abres la página del día (itinerario) y anotas el número del boleto (reserva) en la lista de ese día. Si ya estaba anotado, no lo repites.

33.4 Quitar una reserva: POST /itineraries/{id}/remove-booking/{booking_id} (líneas 81–91)

Es el espejo del anterior: verifica que `booking_ids` exista y que la reserva esté dentro, y la **elimina** de la lista con `.remove(booking_id)`. Guarda y confirma.

33.5 Listar las reservas del plan: GET /itineraries/{id}/bookings (líneas 93–117)

```

@router.get("/{itinerary_id}/bookings")
def get_itinerary_bookings(itinerary_id: int, db: Session = Depends(
    get_db), current_user = Depends(get_current_user)):
    itinerary = db.query(Itinerary).filter(Itinerary.id == itinerary_id)
    .first()
    ...

```

```

booking_ids = (itinerary.route_data or {}).get("booking_ids", [])
if not booking_ids:
    return []
bookings = db.query(Booking).filter(Booking.id.in_(booking_ids)).all()
result = []
for b in bookings:
    service = db.query(Service).filter(Service.id == b.service_id).first()
    result.append({
        "id": b.id, "service_id": b.service_id,
        "service_name": service.name if service else "Desconocido",
        "service_type": service.type if service else "",
        "service_location": service.location if service else "",
        "date": str(b.date), "status": b.status, "total": b.total
    })
return result

```

Paso a paso:

1. Verifica que el itinerario exista y sea tuyo.
2. `(itinerary.route_data or {}).get("booking_ids", [])`: lee la lista de ids de reservas del JSON. Si `route_data` es `None` o no tiene la clave, usa lista vacía (or `{}` y `get(...)` con valor por defecto = “plan seguro”).
3. Si no hay reservas, devuelve lista vacía.
4. Busca **todas las reservas** con esos ids de una vez (`.in_()`).
5. Para cada reserva, busca **su servicio** y arma un diccionario **combinado** con datos de ambas tablas.

33.5.1. El “join” a mano (la idea importante)

Este código hace algo que en SQL se llama **JOIN** (unir tablas): junta la información de `bookings` (fecha, estado, total) con la de `services` (nombre, tipo, ubicación) para darle al frontend una **respuesta lista para mostrar**.

¿Por qué a mano y no con SQL? Es más fácil de leer y controlar para un proyecto chico. En sistemas grandes, SQLAlchemy tiene herramientas para hacer joins automáticos, pero aquí se hace explícito: **un bucle + una búsqueda por cada reserva**.

33.6 Calcular rutas: POST `/itineraries/{id}/directions` (líneas 119–131)

```

@router.post("/{itinerary_id}/directions")
def calculate_directions(itinerary_id: int, req: DirectionsRequest,
    current_user = Depends(get_current_user)):
    itinerary = db.query(Itinerary).filter(Itinerary.id == itinerary_id).first()
    if not itinerary or itinerary.tourist_id != current_user["id"]:
        raise HTTPException(status_code=403, detail="No autorizado")
    result = get_directions(
        origin_lat=req.origin.lat,
        origin_lng=req.origin.lng,
        dest_lat=req.destination.lat,
        dest_lng=req.destination.lng,

```

```

        waypoints=[{"lat": w.lat, "lng": w.lng, "name": w.name} for w in
                    req.waypoints]
    )
    return result

```

La conexión con el mapa. Esta ruta:

1. Verifica que el itinerario sea tuyo (**403** si no).
2. Recibe el `DirectionsRequest` (el formulario anidado de `schemas/itinerary.py`: `origin`, `destination`, `waypoints`).
3. **Extrae** los datos de cada punto (`lat`, `lng`, `name`) y se los pasa a `get_directions` de `maps_utils.py`.
4. Devuelve lo que `get_directions` calcule (la plantilla `DirectionsResponse` que vimos en `schemas`).

¿Quién es `get_directions`? Es una función de `maps_utils.py` que aún no hemos visto, pero ya sabes su trabajo: **hablar con Google Maps** para calcular la ruta entre los puntos. Lo veremos pronto.

Resumen mental

`itineraries.py` = CRUD del plan + **manipular el JSON** (agregar/quitar reservas) + **join a mano** (reservas con sus servicios) + **punto a Google Maps** (`get_directions`).

34. routes/messages.py — los mensajes HTTP (47 líneas)

34.1 ¿Qué hace este archivo?

Gestiona los mensajes **por HTTP** (peticiones normales). Recuerda que el chat en tiempo real vivía en `ws.py` (WebSockets); este archivo es la **versión clásica**: enviar un mensaje y leer el historial de una reserva.

34.2 Enviar mensaje: POST /messages (líneas 12–32)

```

@router.post("/")
def send_message(message: MessageCreate, db: Session = Depends(get_db),
                 current_user = Depends(get_current_user)):
    booking = db.query(Booking).filter(Booking.id == message.booking_id)
    .first()
    if not booking:
        raise HTTPException(status_code=404, detail="Reserva no
                            encontrada")
    service = db.query(Service).filter(Service.id == booking.service_id)
    .first()
    if current_user["id"] != booking.tourist_id and current_user["id"]
    != service.provider_id:
        raise HTTPException(status_code=403, detail="No autorizado")
    new_message = Message(
        sender_id=current_user["id"],
        receiver_id=message.receiver_id,
        booking_id=message.booking_id,
        content=message.content
    )

```

```
db.add(new_message); db.commit(); db.refresh(new_message)
return new_message
```

Paso a paso:

1. Recibe MessageCreate (los 3 campos que vimos: receiver_id, booking_id, content).
2. Busca la **reserva**; si no existe, **404**.
3. **Control de acceso**: solo el turista de la reserva o el prestador del servicio pueden escribir. (Igual que en bookings.py.) Si no, **403**.
4. Crea el Message con sender_id=current_user["id"]: **el remitente lo pone el sistema**, no el cliente. (¡El formulario no incluía sender_id por eso!)
5. Guarda, refresca y devuelve.

34.3 Leer el historial: GET /messages/booking/{booking_id} (líneas 34–47)

```
@router.get("/booking/{booking_id}")
def get_messages(booking_id: int, db: Session = Depends(get_db),
current_user = Depends(get_current_user)):
    booking = db.query(Booking).filter(Booking.id == booking_id).first()
    if not booking:
        raise HTTPException(status_code=404, detail="Reserva no
            encontrada")
    service = db.query(Service).filter(Service.id == booking.service_id)
        .first()
    if current_user["id"] not in [booking.tourist_id, service.
        provider_id]:
        raise HTTPException(status_code=403, detail="No autorizado")
    messages = db.query(Message).filter(Message.booking_id == booking_id)
        .order_by(Message.timestamp).all()
    return messages
```

Paso a paso:

1. Busca la reserva (**404** si no existe).
2. **Control de acceso** con una variante elegante: current_user["id"] not in [booking.tourist_id, service.provider_id] → “¿eres el turista o el prestador?” Si no, **403**.
3. Busca los mensajes de esa reserva y los **ordena por fecha** con order_by sobre Message.timestamp: así el historial sale en orden cronológico.

34.3.1. La novedad: order_by

db.query(...).order_by(Message.timestamp) = “devuélveme los mensajes **ordenados** de menor a mayor fecha” (del más antiguo al más nuevo). El frontend los muestra tal cual, sin tener que ordenarlos él.

34.4 Mensajes HTTP vs WebSockets (recapitulemos)

Aspecto	HTTP (messages.py)	WebSocket (ws.py)
Conexión	Se abre y cierra en cada petición	Se mantiene abierta (tiempo real)
Uso	Enviar un mensaje, leer historial	Chat en vivo, notificaciones al instante
Modelo de datos	Mismo Message	Mismo Message
Guardado	En la base de datos	En la base de datos

Ambos comparten el mismo modelo **Message** y guardan en la misma tabla. La diferencia es **cómo viaja la información**: HTTP es “pide y responde”; WebSocket es “canal abierto”.

Resumen mental

messages.py = la versión **HTTP** del chat. Control de acceso (turista o prestador), sender_id lo pone el sistema, e historial **ordenado por fecha**.

35. routes/reviews.py — las calificaciones (59 líneas)

35.1 ¿Qué hace este archivo?

Gestiona las reseñas con **3 reglas de negocio** muy típicas:

1. Solo quien **viajó de verdad** puede calificar (reserva **confirmada**).
2. **Una reserva = una calificación** (no se puede calificar dos veces).
3. La nota debe estar **entre 1 y 5**.

Y además, al calificar, **actualiza el rating del servicio** (el promedio).

35.2 Crear reseña: POST /reviews (líneas 13–50)

```
@router.post("/")
def create_review(review: ReviewCreate, db: Session = Depends(get_db),
current_user = Depends(get_current_user)):
    if current_user["role"] != "turista":
        raise HTTPException(status_code=403, detail="Solo turistas
            pueden calificar")
    booking = db.query(Booking).filter(Booking.id == review.booking_id).
        first()
    if not booking:
        raise HTTPException(status_code=404, detail="Reserva no
            encontrada")
    if booking.tourist_id != current_user["id"]:
        raise HTTPException(status_code=403, detail="No autorizado")
    if booking.status != "confirmada":
        raise HTTPException(status_code=400, detail="Solo se puede
            calificar una reserva confirmada")
    existing = db.query(Review).filter(Review.booking_id == review.
        booking_id).first()
    if existing:
        raise HTTPException(status_code=400, detail="Ya calificaste esta
            reserva")
    if review.rating < 1 or review.rating > 5:
```

```

        raise HTTPException(status_code=400, detail="La calificación
            debe ser entre 1 y 5")
    new_review = Review(
        booking_id=review.booking_id,
        tourist_id=current_user["id"],
        service_id=booking.service_id,
        rating=review.rating,
        comment=review.comment
    )
    db.add(new_review); db.commit(); db.refresh(new_review)
    avg_rating = db.query(func.avg(Review.rating)).filter(Review.
        service_id == booking.service_id).scalar()
    service = db.query(Service).filter(Service.id == booking.service_id)
        .first()
    service.rating = round(avg_rating, 2)
    db.commit()
    return new_review

```

35.2.1. La escalera de validaciones

La función se puede leer como una **escalera**: sube de a un escalón, y si algo falla, lanza al usuario con el código adecuado:

- **403** ¿No eres turista? No puedes calificar.
- **404** ¿La reserva no existe? No hay nada que calificar.
- **403** ¿La reserva no es tuya? No puedes calificar reservas ajenas.
- **400** ¿La reserva no está confirmada? Solo se califica un viaje hecho.
- **400** ¿Ya calificaste esa reserva? Una reseña por reserva.
- **400** ¿La nota no está entre 1 y 5? Fuera de rango.

Cuando la escalera termina sin caídas, la reseña se guarda.

35.2.2. Actualizar el promedio (líneas 45–48): la parte especial

```

avg_rating = db.query(func.avg(Review.rating)).filter(Review.service_id
    == booking.service_id).scalar()
service = db.query(Service).filter(Service.id == booking.service_id).
    first()
service.rating = round(avg_rating, 2)
db.commit()

```

La novedad: `func.avg` y `.scalar()`.

- `func.avg(Review.rating)` = “**calcular el promedio**” de la columna `rating` de las reseñas. (SQL AVG.)
- `.scalar()` = “dame el resultado como un número **único**” (el promedio), en vez de una fila completa.
- Luego se **guarda** ese promedio redondeado en la columna `rating` del servicio.

Analogía

Calificar es como **votar por una app**: cada vez que llega un voto nuevo, la app recalcula el promedio de estrellas y lo actualiza en la ficha del servicio.

35.3 Ver reseñas de un servicio: GET /reviews/service/{service_id} (líneas 52–55)

```
@router.get("/service/{service_id}")
def get_service_reviews(service_id: int, db: Session = Depends(get_db)):
    reviews = db.query(Review).filter(Review.service_id == service_id).
        order_by(Review.created_at.desc()).all()
    return reviews
```

Novedad pequeña: `order_by(Review.created_at.desc())`. El `.desc()` significa **descendente**: las reseñas más **nuevas primero** (en `messages.py` era ascendente por defecto, aquí lo invertimos para que el cliente vea lo último arriba). No pide login (**público**): cualquiera puede leer las reseñas de un servicio.

35.4 Ver mis reseñas: GET /reviews/my (líneas 57–59)

```
@router.get("/my")
def get_my_reviews(db: Session = Depends(get_db), current_user = Depends(
    get_current_user)):
    return db.query(Review).filter(Review.tourist_id == current_user["id
        "]).all()
```

Consulta directa: “dame todas las reseñas donde el turista soy yo”. Sin validaciones extra, porque solo filtra por tu propio id.

Resumen mental

`reviews.py` = **reglas de negocio en cascada** (rol, dueño, estado, duplicado, rango) + **promedio automático** con `func.avg` + actualización del **rating** del servicio.

36. routes/costs.py — el calculador de costos (47 líneas)

36.1 ¿Qué hace este archivo?

Es un **calculador**: recibe un grupo de reservas del turista y devuelve el **total** del viaje, el **desglose por categoría** (guía, hotel, etc.) y el **detalle** de cada reserva. **No guarda nada** en la base: es una función pura de cálculo que solo lee datos.

36.2 El cálculo: POST /costs/calculate (líneas 11–47)

```
@router.post("/calculate")
def calculate_costs(body: CostCalculateRequest, db: Session = Depends(
    get_db), current_user = Depends(get_current_user)):
    booking_ids = body.booking_ids
    total = 0.0
    breakdown = {"guia": 0.0, "hotel": 0.0}
    details = []
```



```

for booking_id in booking_ids:
    booking = db.query(Booking).filter(Booking.id == booking_id).
        first()
    if not booking:
        continue
    if booking.tourist_id != current_user["id"]:
        continue
    service = db.query(Service).filter(Service.id == booking.
        service_id).first()
    if not service:
        continue
    category = service.type
    if category not in breakdown:
        breakdown[category] = 0.0
    breakdown[category] += booking.total
    total += booking.total
    details.append({
        "booking_id": booking.id,
        "service_name": service.name,
        "category": category,
        "price": booking.total
    })
return {
    "total": round(total, 2),
    "breakdown": {k: round(v, 2) for k, v in breakdown.items() if v
        > 0},
    "details": details
}

```

36.2.1. Las tres partes del resultado

Parte	Qué es
total	La suma de todo : cada reserva de la lista, su total .
breakdown	El total por categoría (guía, hotel, restaurante...): un diccionario donde la clave es el tipo de servicio y el valor es la suma de ese tipo.
details	Una lista de tickets : cada reserva con nombre del servicio, categoría y precio, lista para mostrar.

36.2.2. El patrón if not ...: continue

El bucle recorre los `booking_ids` y, con `continue`, **se salta** los que no le interesan:

- **404 silencioso**: si la reserva no existe, la **omite** (no lanza error).
- **Filtro de seguridad**: si la reserva no es tuya, la **omite**.
- **Defensa**: si el servicio no existe, la **omite**.

Analogía

`continue` = “**paso al siguiente**”. Es como revisar una lista de compras y tachar solo lo tuyo: si un papel no te corresponde, lo dejas de lado y sigues con el siguiente.

Aquí **no se usa raise** (a diferencia de casi todos los endpoints): el frontend manda una lista de ids y el servidor **simplemente ignora** los que no aplican, en vez de romper todo el cálculo por uno solo malo.

36.2.3. El diccionario breakdown y su inicialización

```
breakdown = {"guia": 0.0, "hotel": 0.0}
...
category = service.type
if category not in breakdown:
    breakdown[category] = 0.0
breakdown[category] += booking.total
```

La idea: cada vez que aparece una categoría nueva, se crea su **acumulador** en 0 y luego se le suma. La primera línea es solo un “arranque” con las dos más comunes; la `if` garantiza que **cualquier categoría** pueda aparecer sin error (`KeyError`).

36.2.4. La respuesta con diccionarios anidados (líneas 43–47)

```
return {
    "total": round(total, 2),
    "breakdown": {k: round(v, 2) for k, v in breakdown.items() if v > 0},
    "details": details
}
```

Dos novedades:

- `round(valor, 2)`: **redondea a 2 decimales** (los centavos). Evita resultados como 123.4000000001 por la precisión de los flotantes.
- `{k: round(v, 2) for k, v in breakdown.items() if v > 0}`: un **diccionario por comprensión** con `if` (lo vimos en `schemas/cost.py`): “para cada clave y valor del desglose, redondéalo y **queda solo si es mayor que 0**”, para no mandar categorías vacías.

Resumen mental

`costs.py` = **calculador puro**: lee reservas tuyas, suma por categoría y devuelve **total + breakdown + details**. Usa `continue` en vez de `raise` para **ignorar** lo que no aplica.

37. routes/admin.py — el panel del administrador (39 líneas)

37.1 ¿Qué hace este archivo?

Es el **panel de control** de la plataforma. Recuerda que, cuando alguien se registra como prestador, su cuenta queda **desactivada** hasta que un **admin** la apruebe (el campo `approved` del modelo `User`, que vimos antes, empieza en `False`).

Este archivo le da al admin:

1. Ver la **lista de prestadores pendientes** (los que esperan aprobación).
2. **Aprobar** a un prestador (su cuenta se activa).

3. Rechazar a un prestador (su cuenta se **elimina**).

Aquí aparece el patrón más importante del archivo: **una guardia extra** llamada `require_admin`.

37.2 La guardia: `require_admin` (líneas 9–12)

```
def require_admin(current_user: dict = Depends(get_current_user)):
    if current_user.get("role") != "admin":
        raise HTTPException(status_code=403, detail="Se requiere rol de
            administrador")
    return current_user
```

Una dependencia que protege a otras. Hasta ahora, cada endpoint usaba `get_current_user` (que verificaba “tienes token”). Pero **tener token no significa ser admin**: cualquiera puede loguearse.

`require_admin` hace lo siguiente:

1. Usa `get_current_user` (ya lo conocemos: verifica que haya un usuario logueado).
2. **Comprueba el rol**: si `current_user.get("role")` no es "admin", lanza **403** (“No estás autorizado”).
3. Si es admin, **devuelve el usuario** para que el endpoint lo use.

La idea clave: cada endpoint de este archivo escribe `admin=Depends(require_admin)` en vez de `current_user=Depends(get_current_user)`. Así, **FastAPI ejecuta la guardia primero**, y si no eres admin, el endpoint **ni siquiera se ejecuta**.

Analogía

Imagina que la app es una oficina con dos puertas:

- `get_current_user` = el **guardia de la puerta principal**: verifica que tengas **carnet** (token).
- `require_admin` = un **segundo guardia** en la puerta de la sala de jefes: además del carnet, revisa que tu carnet diga “ADMIN”.

El segundo guardia **ya sabe** que pasaste el primero (lo llama él mismo). Por eso los endpoints no repiten la verificación: **si llegaste a la sala, es porque eres admin**.

La palabra `get`: `current_user.get("role")` es otra forma de leer un valor de un diccionario. A diferencia de `current_user["role"]`, si la clave no existe **no da error**: devuelve `None`. (Un plan “a prueba de balas”.)

37.3 La lista de espera: GET `/prestadores-pendientes` (líneas 14–17)

```
@router.get("/prestadores-pendientes")
def get_pending_prestadores(admin=Depends(require_admin), db: Session =
    Depends(get_db)):
    pending = db.query(User).filter(User.role == "prestador", User.
        approved == False).all()
    return [{"id": u.id, "name": u.name, "email": u.email, "created_at":
        str(u.created_at)} for u in pending]
```

La novedad: el filtro doble. `.filter(..., ...)` con **coma** significa “Y ambas condiciones” (es un AND de SQL):

- `User.role == "prestador"`: solo prestadores.
- `User.approved == False`: solo los **aún no aprobados**.

Juntas: “dame a los prestadores que **esperan** aprobación”.

La otra novedad: la respuesta es una lista de diccionarios, no los objetos `User`. Es un “resumen”:

- Se **eligen** solo 4 campos: `id`, `name`, `email`, `created_at` (para saber desde cuándo espera).
- **No se mandan** datos que el admin no necesita ver (¡como la `password_hash`! Nunca debe salir del servidor).
- `str(u.created_at)`: la fecha se convierte a texto para que viaje bien por JSON.

Analogía

Es como la **bandeja de solicitudes** de una app: el admin ve solo una tarjetita con nombre, correo y fecha de cada aspirante, **sin el expediente completo**.

37.4 Aprobar: POST `/aprobar/{user_id}` (líneas 19–28)

```
@router.post("/aprobar/{user_id}")
def approve_prestador(user_id: int, admin=Depends(require_admin), db:
    Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Usuario no
            encontrado")
    if user.role != "prestador":
        raise HTTPException(status_code=400, detail="El usuario no es
            prestador")
    user.approved = True
    db.commit()
    return {"message": f"Prestador {user.name} aprobado"}
```

La “escalera” que ya conocemos, en versión corta:

- **404** ¿No existe el usuario? Error.
- **400** ¿Existe pero no es prestador? Error (no puedes aprobar a un turista).
- Si llega al final: `user.approved = True` (**encender la luz**) + `db.commit()` (guardar) + mensaje de confirmación.

El dato nuevo: f-string en el mensaje. `f"Prestador {user.name} aprobado"` es un texto que **inserta valores**: las llaves `{user.name}` se reemplazan por el nombre real. Por eso el frontend puede mostrar “Prestador Juan aprobado”.

37.5 Rechazar: POST `/rechazar/{user_id}` (líneas 30–39)

```
@router.post("/rechazar/{user_id}")
def reject_prestador(user_id: int, admin=Depends(require_admin), db:
    Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="Usuario no
            encontrado")
    if user.role != "prestador":
        raise HTTPException(status_code=400, detail="El usuario no es
            prestador")
    db.delete(user)
    db.commit()
    return {"message": f"Prestador {user.name} rechazado y eliminado"}
```

Igual que aprobar... casi. Las validaciones son idénticas (404 y 400). La diferencia está en el final: en vez de `approved = True`, usa `db.delete(user) + db.commit()`.

37.5.1. ¿Por qué “aprobar” solo marca y “rechazar” borra?

Esta es una decisión de diseño que debes notar:

Acción	Qué pasa con el registro
Aprobar	El usuario sigue existiendo , solo se le pone la “luz verde” (<code>approved=True</code>). Puede volver a <code>False</code> si hiciera falta.
Rechazar	El usuario desaparece de la base (<code>db.delete</code>). La cuenta se borra para siempre.

Analogía

Aprobar es como **sellar un permiso** (el documento queda, con su sello). Rechazar es como **tirar el documento a la basura** (ya no existe).

Ojo con una consecuencia: si el prestador rechazado **ya tenía servicios creados**, al borrar al usuario esos servicios pueden quedarse “huérfanos” (sin dueño). Es un detalle que un sistema real tendría que manejar (por ejemplo, borrar o transferir sus servicios primero). No lo hace aquí, pero está bien saberlo.

Resumen mental

`admin.py` = el **panel del jefe**: una guardia (`require_admin`) protege todo el archivo, filtra por **dos condiciones** (`approved == False`), **elige solo los campos** de la respuesta, y termina con dos poderes: **aprobar** (marcar) o **rechazar** (borrar).

38. routes/ws.py — el chat en tiempo real (107 líneas)

38.1 ¿Qué hace este archivo?

Es el **chat en vivo**: los mensajes llegan **al instante**, sin que el usuario tenga que preguntar. Es el archivo más “distinto” de la carpeta, porque no usa HTTP sino **WebSockets**. Además marca en la base de datos si un mensaje fue **leído**.

38.2 Primero, lo importante: ¿qué es un WebSocket?

Hasta ahora, todo viajaba por HTTP. WebSocket es otro **canal** de comunicación:

Aspecto	HTTP	WebSocket
Se parece a	Enviar una carta	Hablar por teléfono
Conexión	Se abre, se responde y se cierra	Queda abierta todo el tiempo
Comunicación	Pregunta → respuesta	El servidor puede empujar mensajes solo
Ejemplo	GET /services	Chat en vivo, notificaciones

La clave: con WebSocket el servidor puede **enviar solo** (sin que le pregunten). Cuando A envía un mensaje a B, el servidor lo recibe, lo guarda y **empuja** el mensaje hacia B al instante. Eso no se puede hacer con HTTP normal.

38.3 El ConnectionManager: la recepcionista (líneas 12–33)

```
class ConnectionManager:
    def __init__(self):
        self.active_connections: dict[int, list[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, user_id: int):
        await websocket.accept()
        if user_id not in self.active_connections:
            self.active_connections[user_id] = []
        self.active_connections[user_id].append(websocket)

    def disconnect(self, websocket: WebSocket, user_id: int):
        if user_id in self.active_connections:
            self.active_connections[user_id].remove(websocket)
            if not self.active_connections[user_id]:
                del self.active_connections[user_id]

    async def send_to_user(self, user_id: int, message: dict):
        if user_id in self.active_connections:
            for conn in self.active_connections[user_id]:
                await conn.send_json(message)

manager = ConnectionManager()
```

¿Qué es este “diccionario de listas”? `active_connections` guarda **quién está conectado** y en qué canal:

- **Clave** = `user_id` (el número del usuario).
- **Valor** = una **lista** de conexiones abiertas de ese usuario (porque un mismo usuario puede estar en el celular y en la computadora al mismo tiempo).

Analogía

ConnectionManager es la **recepcionista de un hotel** con un cuaderno:

- **connect**: el huésped llega, la recepcionista lo **acepta** y apunta “Habitación 5: Juan” (lo agrega a la lista).
- **disconnect**: el huésped se va, lo **borra del cuaderno** (y si era el único en esa habitación, borra la línea completa).
- **send_to_user**: la recepcionista **corre a entregar** un mensaje a **todas** las conexiones de ese huésped.

La línea `manager = ConnectionManager()` (33) es **muy importante**: es una **única recepcionista para todos**. Como se crea una sola vez (fuera de la función), todos los usuarios conectados comparten el **mismo cuaderno**. Si cada conexión tuviera su propio manager, nadie sabría quién está conectado.

38.4 El canal WebSocket (líneas 35–107)

```
@router.websocket("/ws/chat/{user_id}")
async def websocket_chat(websocket: WebSocket, user_id: int):
    await manager.connect(websocket, user_id)
    try:
        while True:
            data = await websocket.receive_json()
            action = data.get("action")
            if action == "send_message":
                ...
            elif action == "mark_read":
                ...
    except WebSocketDisconnect:
        manager.disconnect(websocket, user_id)
```

Paso a paso del canal:

1. El usuario entra a `/ws/chat/{user_id}`: aquí el id va **directo en la URL** (¡nota que no hay token ni Depends! Es un mundo aparte del HTTP).
2. `await manager.connect(...)`: acepta la conexión y lo apunta en el cuaderno.
3. **El bucle while True**: el canal **queda abierto y escuchando**. Cada vuelta **espera** el próximo mensaje que llegue.
4. **action**: es el “**botón del menú**” que viaja dentro del mensaje. Hay dos: `send_message` (enviar) y `mark_read` (marcar leído).
5. Si el cliente se desconecta, salta `WebSocketDisconnect` y se ejecuta `manager.disconnect`: la recepcionista **lo borra del cuaderno**.

Una pausa sobre async y await: son la forma que usa Python para “no bloquearse”. `await` significa “**espera aquí** hasta que llegue la respuesta, pero mientras tanto haz otras cosas”. Es como pedir la comida y **platicar** con un amigo mientras llega, en vez de quedarte mirando la cocina. Por eso un servidor con muchos canales abiertos no se congela.

38.4.1. Acción send_message: enviar (líneas 43–89)

```
if action == "send_message":
    booking_id = data["booking_id"]
    receiver_id = data["receiver_id"]
    content = data["content"]
    db = next(get_db())
    try:
        booking = db.query(Booking).filter(Booking.id == booking_id).
            first()
        if not booking:
            await websocket.send_json({"error": "Reserva no encontrada"
            })
            continue
        service = db.query(Service).filter(Service.id == booking.
            service_id).first()
        if user_id != booking.tourist_id and user_id != service.
            provider_id:
            await websocket.send_json({"error": "No autorizado"})
            continue
        new_message = Message(
            sender_id=user_id, receiver_id=receiver_id,
            booking_id=booking_id, content=content
        )
        db.add(new_message); db.commit(); db.refresh(new_message)
        message_data = {
            "id": new_message.id, "sender_id": new_message.sender_id,
            "receiver_id": new_message.receiver_id, "booking_id":
                new_message.booking_id,
            "content": new_message.content, "timestamp": str(new_message
                .timestamp)
        }
        await manager.send_to_user(receiver_id, {"action": "new_message"
            , "message": message_data})
        await websocket.send_json({"action": "message_sent", "message":
            message_data})
    finally:
        db.close()
```

Es el mismo “enviar mensaje” de messages.py, pero por canal abierto:

1. Lee del mensaje: a qué **reserva**, a **quién** y **qué** dice.
2. **Valida** (igual que en HTTP): ¿existe la reserva? ¿eres el turista o el prestador? Si falla, responde **por el mismo canal** con un error y continue (“sigue escuchando, no cierras”).
3. Guarda el Message en la base (igual que siempre).
4. **Lo importante, el doble envío:**
 - `send_to_user(receiver_id, ...)`: la recepcionista **empuja el mensaje al receptor si está conectado**. Así llega al instante.
 - `send_json(...)` al remitente: le confirma “**mensaje enviado**”.

La diferencia con HTTP en una frase: en messages.py el receptor tiene que **preguntar** por los mensajes; aquí el servidor **los empuja solos**.

38.4.2. La base de datos “a mano”: next(get_db()) y finally

En HTTP, FastAPI **inyecta** la sesión con Depends(get_db). En WebSocket ese patrón no aplica igual, así que aquí la abren **manualmente**:

- `db = next(get_db())` = “toma la llave de la puerta de la base”.
- `finally: db.close()` = “siempre devuelve la llave, pase lo que pase” (haya error o no).

Analogía

Es como **prestar un libro**: lo sacas (`next`), haces lo que necesitas (tu código), y con `finally` te aseguras de **devolverlo siempre**, aunque se te haya caído el café en el camino.

38.4.3. Acción mark_read: marcar leído (líneas 91–104)

```
elif action == "mark_read":
    booking_id = data["booking_id"]
    db = next(get_db())
    try:
        db.query(Message).filter(
            Message.booking_id == booking_id,
            Message.receiver_id == user_id,
            Message.read == False
        ).update({"read": True})
        db.commit()
        await websocket.send_json({"action": "read_confirmed", "
            booking_id": booking_id})
    finally:
        db.close()
```

Una actualización en lote:

- **Filtro de 3 condiciones** (todas “Y”): mensajes de esa reserva, dirigidos a mí (`receiver_id == user_id`) y que **aún no estén leídos**.
- `.update({"read": True})`: **todos esos a la vez** pasan a “leídos”, de una sola instrucción (sin bucle, sin buscar uno por uno).
- Confirma por el canal con `read_confirmed`.

¿Por qué hace falta? Cuando abres el chat, el frontend avisa “marca como leído lo que es mío”, y el remitente puede ver si su mensaje ya fue visto. Así, sin recargar la página.

Resumen mental

`ws.py` = el **chat en tiempo real**: una **repcionista** (`ConnectionManager`) que anota quién está conectado, un **canal abierto** (`while True`) que escucha y **empuja** mensajes al instante, y dos comandos: `send_message` y `mark_read`.

38.5 ¡Fin de routes/! El mapa de la carpeta

Con `ws.py` termina toda la carpeta `routes/`. Mapa rápido de los 9 archivos:

Archivo	En una frase
auth.py	Registro, verificación, login y tokens . Aquí nace <code>get_current_user</code> , el “guardián” que usa todo el resto.
services.py	El catálogo : crear, ver, buscar y editar servicios.
bookings.py	Las reservas y sus reglas (quién puede, cuándo).
itineraries.py	Los planes de viaje : JSON, rutas con Google Maps.
messages.py	El chat por HTTP (enviar y leer historial).
reviews.py	Las calificaciones con sus reglas y el promedio.
costs.py	El calculador de costos del viaje.
admin.py	El panel del admin : aprobar y rechazar prestadores.
ws.py	El chat en tiempo real con WebSockets.

El hilo que los une: todos se apoyan en 4 bloques que ya dominas: el **db** (base de datos), los **modelos** (las tablas), los **schemas** (los formularios) y `get_current_user` (el carnet). Los 9 archivos son variaciones de ese mismo esqueleto.

39. email_utils.py — el cartero (52 líneas)

39.1 ¿Qué hace este archivo?

Hasta ahora vimos **rutas** (que reciben peticiones). Este es otro tipo de archivo: una **utilidad** (de ahí “utils”). No tiene endpoints ni responde peticiones: es un **servicio interno** que otros archivos **llaman** cuando lo necesitan.

Concretamente, se encarga de **enviar el correo de verificación** con el código de 6 dígitos que viste en `auth.py`. ¿Te acuerdas del “EMAIL SIMULATED” que aparecía en consola al registrarte? **Se genera aquí**.

Analogía

Las rutas son la **tienda** (atienden al cliente). `email_utils.py` es la **cocina**: nadie entra a la cocina por la puerta de la tienda; la tienda le **pide** a la cocina el pastel (`send_verification_email`) y la cocina lo hace.

39.2 La configuración del cartero (líneas 8–12)

```
SMTP_HOST = os.getenv("SMTP_HOST", "smtp.gmail.com")
SMTP_PORT = int(os.getenv("SMTP_PORT", "587"))
SMTP_USER = os.getenv("SMTP_USER", "")
SMTP_PASSWORD = os.getenv("SMTP_PASSWORD", "")
SMTP_FROM = os.getenv("SMTP_FROM", SMTP_USER)
```

El mismo patrón de `config.py` que ya conoces: `os.getenv` con un valor por defecto. Aquí se configura el **servidor de correo** (`smtp` = “protocolo para enviar correo”):

- `SMTP_HOST`: la dirección del servidor (por defecto, el de **Gmail**).
- `SMTP_PORT`: el “**número de puerta**” (587 = la puerta de correo común con cifrado).
- `SMTP_USER` y `SMTP_PASSWORD`: el **usuario y contraseña** del correo que envía. Por defecto **vacíos** (¡importante!).
- `SMTP_FROM`: **quién aparece como remitente** (por defecto, el propio usuario).

39.3 El código de 6 dígitos (líneas 14–15)

```
def generate_verification_code() -> str:
    return str(random.randint(100000, 999999))
```

Genera un número al azar entre 100000 y 999999 (siempre 6 dígitos) y lo devuelve como texto (por eso `str(...)`). `random` es el módulo de Python para azar, como una **ruleta de casino** para números.

La flecha `->str` dice: “esta función devuelve un texto”. Es una **pista** de qué esperar (ayuda a leer, no obliga nada).

39.4 El envío real: `_send_email` (líneas 17–49)

```
def _send_email(to_email: str, code: str):
    try:
        if not SMTP_USER or not SMTP_PASSWORD:
            print(f"[EMAIL SIMULATED] To: {to_email}, Code: {code}")
            return
        msg = MIMEMultipart()
        msg["From"] = SMTP_FROM
        msg["To"] = to_email
        msg["Subject"] = "TravelHub - Código de verificación"
        body = f""" <html> ... Tu código de verificación es: {code} ...
            </html> """
        msg.attach(MIMEText(body, "html"))
        with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
            server.starttls()
            server.login(SMTP_USER, SMTP_PASSWORD)
            server.send_message(msg)
        print(f"[EMAIL SENT] To: {to_email}")
    except Exception as e:
        print(f"[EMAIL FAILED] To: {to_email}, Error: {e}")
```

Nota: el guion bajo al inicio (`_send_email`) es una **convención**: significa “esta función es interna, no la llames desde fuera”. Es como una habitación marcada “solo empleados”.

39.4.1. El modo simulado (líneas 19–21): el secreto para desarrolladores

```
if not SMTP_USER or not SMTP_PASSWORD:
    print(f"[EMAIL SIMULATED] To: {to_email}, Code: {code}")
    return
```

Si las credenciales están **vacías** (como al principio, sin configurar), **no intenta enviar**: solo **imprime el código en consola** y **return** (termina).

¿Por qué es genial? Puedes **probar todo el flujo de registro sin un correo real**: ves el código en la consola y lo usas para verificar. Sin esta trampilla, desarrollar sería imposible (tendrías que configurar Gmail de verdad cada vez).

Analogía

Es el **teléfono de juguete** para ensayar la obra de teatro: haces toda la actuación (registro, código, verificación) sin necesidad del teatro de verdad. Cuando llegue el día del estreno (credenciales reales), el mismo guion funciona.

39.4.2. Construir el correo (líneas 23–40)

```
msg = MIMEMultipart()
msg["From"] = SMTP_FROM
msg["To"] = to_email
msg["Subject"] = "TravelHub - Código de verificación"
body = f""" <html> ... {code} ... </html> """
msg.attach(MIMEText(body, "html"))
```

Un correo se arma en **dos partes**:

1. **El sobre** (MIMEMultipart): lleva From, To y Subject (quién, para quién, asunto).
2. **La carta adentro** (msg.attach): el **HTML** con el diseño bonito (colores, el código gigante en el centro). MIMEText(body, "html") dice “este texto es HTML, no texto plano”.

La carta es un f-string enorme: por eso {code} se reemplaza por el número real. (La parte HTML *real* del archivo tiene más etiquetas: títulos, colores, espaciado; aquí la abrevié para que se vea la idea.)

39.4.3. Entregar el correo (líneas 42–45)

```
with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
    server.starttls()
    server.login(SMTP_USER, SMTP_PASSWORD)
    server.send_message(msg)
```

Tres pasos para entregar:

1. starttls(): **activa el cifrado** (sella la carta para que nadie la lea en el camino).
2. login(...): **identificarse** ante el servidor con usuario y contraseña.
3. send_message(msg): **entregar** el correo.

El with es tu amigo: cuando termina el bloque (o hay un error), el servidor se **cierra solo**. Es el mismo “devolver la llave siempre” que viste en `ws.py`.

39.4.4. La red de seguridad (líneas 48–49)

```
except Exception as e:
    print(f"[EMAIL FAILED] To: {to_email}, Error: {e}")
```

Si **cualquier cosa** falla (no hay internet, la contraseña es mala, el servidor está caído), se **captura** y solo se **imprime el error**: la **app** no se rompe.

Analogía

Es el **colchón de seguridad**: el correo puede fallar, pero el usuario **igual puede registrarse** (el registro no depende del correo). Fallar en silencio y seguir es mejor que **estrellar toda la app** por un correo que no salió.

39.5 La función que todos usan: `send_verification_email` (líneas 51–52)

```
def send_verification_email(to_email: str, code: str):
    threading.Thread(target=_send_email, args=(to_email, code), daemon=
        True).start()
```

Es la función **pública** (sin guion bajo) que `auth.py` llama. Su magia: **lanza el envío en un hilo aparte** (`threading.Thread`).

¿Por qué en un hilo? Enviar un correo **toma tiempo** (abrir conexión, login, red...). Si lo hiciera en línea, el usuario **esperaría congelado** en el registro mientras el correo se envía. Con un **hilo**, el correo se envía “**por detrás**” mientras la app **sigue atendiendo** al usuario.

Analogía

Es como darle la carta a un **asistente** para que la lleve al correo mientras tú sigues **conversando con el cliente**. No lo haces esperar a que vuelva el asistente.

El `daemon=True` es el último detalle: dice “este ayudante es **de confianza**, no bloquea el cierre de la tienda”. Si la app se apaga, el hilo se apaga con ella sin dejarla esperando.

Resumen mental

`email_utils.py` = el **cartero**: genera el código de 6 dígitos, arma el correo (envelope + HTML), lo envía cifrado, y tiene **modo simulado** (sin credenciales imprime el código en consola) + **red de seguridad** (nunca rompe la app) + **hilos** (no hace esperar al usuario).

40. `maps_utils.py` — el navegador (112 líneas)

40.1 ¿Qué hace este archivo?

Otra **utilidad** (como `email_utils.py`): no tiene rutas, es un servicio que otros archivos llaman. Es la función que `itineraries.py` invocaba en `POST /itineraries/{id}/directions`: **calcular la ruta** entre el origen, los puntos intermedios y el destino.

Pero aquí está la idea más valiosa de todo el archivo: tiene **dos planes**:

Plan	Cuándo se usa
Google Maps (<code>get_directions</code>)	Cuando hay API key configurada y Google responde bien.
Estimación (<code>_estimate_directions</code>)	Plan B : sin API key, o si Google falla o responde mal.

Analogía

Es como **llamar a un taxi** para ir a un lugar: si el taxi llega (**Google**), el viaje es exacto. Pero si no hay taxi (sin clave) o el taxi se descompone (error), **caminas con un plano** y estimas la distancia **al vuelo**. **Siempre llegas** a un resultado.

40.2 Primero, ¿qué es una “API key”?

Una **API** es “la cocina de otra persona”: tú no sabes cómo Google calcula rutas, solo le **pides** y te responde. La **key** es la **credencial** (como tu carnet de acceso) que te da derecho a usar ese servicio (y Google te **cobra** por usarlo).

Por eso, en desarrollo, la clave suele estar **vacía** (como SMTP en el cartero): para no gastar dinero ni configurar cuentas. Y de ahí nace el **plan B**.

40.3 El plan A: get_directions (líneas 4–62)

```
def get_directions(origin_lat, origin_lng, dest_lat, dest_lng, waypoints
=None):
    if not GOOGLE_MAPS_API_KEY:
        return _estimate_directions(...)
    origin = f"{origin_lat},{origin_lng}"
    destination = f"{dest_lat},{dest_lng}"
    wp = ""
    if waypoints:
        wp = "|".join(f"{p['lat']},{p['lng']}" for p in waypoints)
    url = "https://maps.googleapis.com/maps/api/directions/json"
    params = {"origin": origin, "destination": destination,
              "key": GOOGLE_MAPS_API_KEY, "mode": "driving",
              "units": "metric", "language": "es"}

    if wp:
        params["waypoints"] = wp
    try:
        resp = requests.get(url, params=params, timeout=10)
        data = resp.json()
        if data["status"] != "OK":
            return _estimate_directions(...)
        ...
    except Exception:
        return _estimate_directions(...)
```

El camino del plan A, paso a paso:

1. **Chequeo rápido** (líneas 7–8): si no hay API key, **no pierde tiempo** y va directo al plan B.
2. **Prepara las coordenadas** (líneas 10–14): cada punto se convierte a texto “latitud,longitud”. Los puntos intermedios se **unen con una barra** (“|”.join(...)), que es el formato que espera Google.
3. **Pregunta a Google** (líneas 16–26): construye la **URL** del servicio de direcciones y un diccionario **params** con la pregunta: **de dónde** (origin), **a dónde** (destination), **clave**, **en auto** (mode=driving), **en kilómetros** (units=metric) y en **español** (language=es).
4. **Envía y espera** (línea 29): `requests.get(...)` es “yo pregunto, Google responde”. Igual que una ruta GET, pero hacia el servidor de Google.
5. **Lee la respuesta** (línea 30): `resp.json()` convierte el JSON que devolvió Google en un diccionario de Python.
6. **¿Respuesta buena?** (línea 31): si `status != "OK"` (Google dice “no encontré ruta”), **plan B**.
7. **Si todo bien:** procesa la respuesta (lo vemos a continuación).

8. **La red de seguridad** (línea 61–62): si **cualquier** error ocurre (sin internet, tiempo agotado...), **plan B**. Igual que el cartero: nunca rompe la app.

Fíjate cómo funciona el “plan B” en la práctica: `_estimate_directions` se llama desde **3 lugares** (sin clave, respuesta mala, error). Es el mismo “reparador de emergencias” de siempre.

40.3.1. Procesar la respuesta buena de Google (líneas 34–60)

```
legs = []
total_distance_m = 0
total_duration_s = 0
for leg in data["routes"][0]["legs"]:
    legs.append({
        "distance_m": leg["distance"]["value"],
        "distance_text": leg["distance"]["text"],
        "duration_s": leg["duration"]["value"],
        "duration_text": leg["duration"]["text"],
        "start_address": leg.get("start_address", ""),
        "end_address": leg.get("end_address", "")
    })
    total_distance_m += leg["distance"]["value"]
    total_duration_s += leg["duration"]["value"]
polyline = data["routes"][0].get("overview_polyline", {}).get("points",
    "")
return {
    "total_distance_m": total_distance_m,
    "total_distance_km": round(total_distance_m / 1000, 2),
    "total_duration_s": total_duration_s,
    "total_duration_min": round(total_duration_s / 60, 1),
    "total_duration_text": f"{total_duration_s // 60} min",
    "legs": legs,
    "polyline": polyline,
    "source": "google"
}
```

¿Qué es un “leg”? Es **cada tramo** del viaje: del origen al primer punto, de ese punto al siguiente... Google devuelve **una lista de tramos** y este código:

1. Los **recorre uno por uno** y guarda de cada uno: **distancia** (en metros y en texto “12,4 km”), **duración** (en segundos y en texto), y las **direcciones** de inicio y fin.
2. Va **sumando** distancia y duración totales.
3. Extrae el **polyline**: un texto cifrado con la **forma del recorrido**, que el frontend usa para **dibujar la línea en el mapa**.
4. Devuelve el **resumen final**: totales + lista de tramos + polyline + “source”: “google” (“lo calculó Google de verdad”).

¿Te suena esa forma? Es la plantilla `DirectionsResponse` que vimos en la carpeta `schemas` (en `itinerary.py`): **este diccionario es lo que se llena al final**. Conecta con lo que estudiaste en `schemas`.

40.4 El plan B: `_estimate_directions` (líneas 65–112)

```

def _estimate_directions(...):
    from math import radians, sin, cos, sqrt, atan2
    def haversine(lat1, lng1, lat2, lng2):
        R = 6371000
        dlat = radians(lat2 - lat1)
        dlng = radians(lng2 - lng1)
        a = sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2)) *
            sin(dlng/2)**2
        c = 2 * atan2(sqrt(a), sqrt(1 - a))
        return R * c
    points = [{"lat": origin_lat, "lng": origin_lng}]
    if waypoints:
        points.extend(waypoints)
    points.append({"lat": dest_lat, "lng": dest_lng})
    total_distance_m = 0
    legs = []
    for i in range(len(points) - 1):
        d = haversine(points[i]["lat"], points[i]["lng"], points[i+1]["lat"], points[i+1]["lng"])
        total_distance_m += d
        legs.append({"distance_m": round(d),
                    "distance_text": f"{round(d/1000, 2)} km" if d >= 1000 else f"{round(d)} m",
                    "duration_s": round(d / 1.4),
                    "duration_text": f"{round(d / 1.4 / 60)} min",
                    "start_address": ..., "end_address": ...})
    speed_ms = 1.4
    total_duration_s = round(total_distance_m / speed_ms)
    return { ... "source": "estimate" }

```

La fórmula de Haversine (líneas 70–76): es una **fórmula matemática** que calcula la distancia **entre dos puntos sobre una esfera** (la Tierra). Por eso el radio $R = 6371000$ (¡la Tierra mide **6.371 km** de radio, en metros!). Devuelve la distancia “**en línea recta sobre el planeta**” (a vuelo de pájaro), sin calles.

Analogía

Google sabe las **calles reales**. La estimación solo mide **la recta en el globo**: es **más corta** que la ruta real (nadie vuela por las calles), pero sirve para **orientarte** mientras no hay Google.

El resto es el mismo esqueleto de siempre:

1. Arma la lista de puntos: origen + intermedios + destino (líneas 78–81).
2. **Recorre pares consecutivos** (origen→punto 1, punto 1→punto 2, ..., último→destino) y calcula cada tramo con Haversine.
3. Estima el **tiempo** con una velocidad fija 1.4 m/s (un paso humano promedio al caminar):
 $\text{tiempo} = \text{distancia} \div \text{velocidad}$.
4. Devuelve el **mismo formato** que Google, pero con polyline vacío y "source": "estimate".

40.5 La lección grande: el fallback (plan B)

Esta es la idea profesional que debes quedarte de este archivo:

Resumen mental

Un **fallback** = “si esto falla, haz esto otro”. `maps_utils.py` nunca se queda sin respuesta: siempre **intenta lo caro y preciso** (Google) y, si no puede, **entrega algo decente** (estimación) **sin romper la app**. Eso se llama **degradación elegante**.

Y el campo `source` es el **detalle de calidad**: como la respuesta dice de dónde vino (“google” o “estimate”), el frontend puede mostrar “ruta exacta” o “estimación”, o el desarrollador puede **depurar** por qué se usó el plan B.

Resumen mental

`maps_utils.py` = el **navegador con dos planes**: pregunta a **Google Maps** (distancias, tiempos, polyline) y si no hay clave o algo falla, usa la **fórmula de Haversine** para **estimar** la distancia. Devuelve **siempre** el mismo formato y marca `source` para saber cuál plan se usó.

41. travelhub.db — la base de datos (el archivo de datos)

41.1 ¿Qué es este archivo?

Todos los archivos que hemos visto son **código**: instrucciones. Este es otro tipo: es **donde viven los datos**. Es la base de datos **SQLite** de la app, en un solo archivo físico.

- **Tipo de archivo**: SQLite (una base de datos **sin servidor**, que vive en **un solo archivo**).
- **Tamaño real en tu proyecto**: 57 KB (¡casi nada!).
- **Quién lo crea**: la **propia app**, automáticamente. Recuerda `database.py`: al arrancar, la app ejecuta `create_all(bind=engine)`, que **crea las tablas** si no existen. Tú no escribes este archivo a mano.

Analogía

El **código** es la **receta** (los modelos dicen “así es una reserva”). La **base de datos** es la **despensa**: ahí se **guardan los ingredientes reales** (las reservas que los usuarios hicieron de verdad).

41.2 ¿Qué hay adentro? Las 6 tablas

Si abrimos el archivo, encontramos **6 tablas**, exactamente las mismas que estudiamos en `models/`:

Tabla	Columnas	Filas
<code>users</code>	<code>id</code> , <code>name</code> , <code>email</code> , <code>password_hash</code> , <code>role</code> , <code>verified</code> , <code>created_at</code>	4
<code>services</code>	<code>id</code> , <code>provider_id</code> , <code>type</code> , <code>name</code> , <code>description</code> , <code>price</code> , <code>location</code> , <code>rating</code> , <code>available</code> , <code>created_at</code>	2
<code>itineraries</code>	<code>id</code> , <code>tourist_id</code> , <code>day</code> , <code>route_data</code> , <code>created_at</code>	1
<code>bookings</code>	<code>id</code> , <code>tourist_id</code> , <code>service_id</code> , <code>date</code> , <code>status</code> , <code>total</code> , <code>created_at</code>	1
<code>messages</code>	<code>id</code> , <code>sender_id</code> , <code>receiver_id</code> , <code>booking_id</code> , <code>content</code> , <code>read</code> , <code>timestamp</code>	1

Tabla	Columnas	Filas
reviews	id, booking_id, tourist_id, service_id, rating, comment, created_at	0

Observa dos cosas:

1. Los nombres y columnas son los que pusimos en los modelos (en SQLAlchemy se llama “la tabla mapea al modelo”): `Bookings` → `bookings`, `User` → `users`, etc.
2. Las 6 tablas existen incluso con 0 filas (mira `reviews`): cuando la app crea la base, **crea todas** las tablas a la vez, estén vacías o no. Las filas actuales (4 usuarios, 2 servicios, etc.) son **datos de prueba** de cuando corriste la app.

41.3 La primera lección: `create_all` no toca lo que ya existe

Dato importante de cómo se crean: `create_all` solo crea las tablas que faltan. Si una tabla ya existe, **no la modifica**. Parece un detalle, pero tiene una consecuencia real que ya puedes ver en tu proyecto:

El modelo <code>User</code> dice	La tabla <code>users</code> real tiene
id, name, email, password_hash, role, verified, created_at approved, verification_code, verification_code_expires	id, name, email, password_hash, role, verified, created_at (no existen)

Es decir: el **código** ya conoce las columnas `approved` y los códigos de verificación (los agregaste al modelo después), pero la **base guardada** se creó **antes** y `create_all` **no la actualizó**. El código y la base están **desincronizados**.

Analogía

Es como si actualizaras la **receta** (el modelo) para incluir “canela”, pero la **despensa** (la base) sigue con el estante de antes: la receta dice que existe, pero en el estante no hay canela.

¿Por qué importa? Si el código ejecuta `user.approved = True` (como hace `admin.py`), la base **fallará**: no existe esa columna. Por eso los proyectos reales usan **migraciones** (una herramienta llamada **Alembic**) que sirve para “**actualizar la despensa de a poco**”: agregan las columnas nuevas sin borrar los datos.

La solución práctica por ahora: borrar el archivo y dejar que la app lo cree de nuevo con la estructura nueva. Como es una base de prueba, es lo más simple. (¡Y por eso poder borrarla sin miedo es parte del diseño!)

41.4 La segunda lección: `*.db` está en `.gitignore`

Si abres `.gitignore`, verás la línea `*.db`. Eso significa que la base **NO se sube a Git**: cada quien, al clonar el proyecto y correr la app, se crea **su propia base local**.

- **Lo bueno:** el código se comparte (recetas), los datos no (cada despensa es local). Evita conflictos entre desarrolladores.
- **El otro lado de la moneda:** los datos de prueba no viajan con el repositorio; cada máquina empieza de cero.

Resumen mental

`travelhub.db` = **donde viven los datos** (no es código). Lo crea la app automáticamente con las **6 tablas** de los modelos. Lección clave: `create_all` solo crea lo que falta (no actualiza lo existente) y `*.db` está en `.gitignore` (los datos no viajan por Git).

42. Procfile — la instrucción para subir la app (1 línea)

42.1 ¿Para qué sirve?

Procfile viene de “**Process File**” (archivo de procesos). Es un archivo **tiny** que usan las plataformas de **hospedaje en la nube** (Heroku, Render, etc.) para saber **cómo iniciar tu app**.

Sin este archivo, la plataforma **no sabría** que tu proyecto es una app web ni cómo arrancarla. Es la **única línea** que tiene:

```
web: uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

42.2 Desmenucemos la línea

web: La palabra antes de los dos puntos es el **tipo de proceso**. “**web**” le dice a la plataforma: “**este es el servidor que recibe el tráfico web**”. Es el único proceso obligatorio para una web.

El resto es **el mismo comando** que ya usas para correr la app localmente (`uvicorn`), pero con dos diferencias para producción:

Pieza	Qué hace
<code>uvicorn app.main:app</code>	Inicia el servidor: “del módulo <code>app.main</code> , usa el objeto <code>app</code> ” (el que FastAPI creó en <code>main.py</code>). Es el mismo arranque que ya conoces.
<code>--host 0.0.0.0</code>	Escucha en todas las direcciones , no solo en tu computadora. Localmente basta <code>127.0.0.1</code> ; en la nube, 0.0.0.0 abre la puerta a todo el internet.
<code>--port \$PORT</code>	El puerto lo asigna la plataforma : <code>\$PORT</code> es una variable de entorno que la nube llena al momento de arrancar (en tu PC usabas 8000 fijo; aquí lo decide el host).

Analogía

Es la **tarjeta que le das al conserje del hotel** para cuando llegues: “habitación para `app.main`, con ventana `0.0.0.0`, y **el número de habitación me lo asignas tú al hacer check-in**” (`$PORT`). El hotel (la nube) la lee y **deja todo listo antes de que llegues**.

42.3 La conexión con render.yaml y main.py

Fíjate que **no estás aprendiendo algo nuevo**: la plataforma Render (que también tienes en el proyecto, en `render.yaml`) tiene esta línea *idéntica* como su `startCommand`:

```
startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Y el propio `main.py` usa el mismo comando al final: `uvicorn.run("app.main:app", host="0.0.0.0", port=8000)`. Es **el mismo arranque en tres lugares**: el Procfile (Heroku), `render.yaml` (Render) y `main.py` (local). Todos dicen lo mismo: “**esto es una app FastAPI y se arranca así**”.

La diferencia del puerto: en local usas **8000** (fijo, tuyo). En la nube, el puerto **cambia con cada despliegue**, así que se lee de `$PORT` en vez de escribirlo a mano.

Resumen mental

Procfile = la **instrucción de arranque** para la nube: “web” + **uvicorn** + “escucha en 0.0.0.0” + “puerto **asignado** por la plataforma (`$PORT`)”. El mismo comando aparece en `render.yaml` y en `main.py`.